GPSDO v1.08 — Kompletna instrukcja obsługi (od zera do locka)

English | **Polski** | Español

📄 PDF: English · Polski · Español

📖 Strona projektu · README · Changelog · Tuner

Firmware: GPSDO v1.08.71rt, autor jmnlabs (na bazie oryginalnego GPSDO André Balsy, pętli PI Larsa Waleniusa, akumulatora wielopoziomowego Alana Cashina i autorskiego filtru Kalmana) **Płytka:** WeAct BlackPill STM32F411CE · **OCXO:** 10 MHz (np. Vectron C4550) **GPS:** odbiornik u-blox na Serial1 (LEA-M8T / NEO-M8T / ZED-F9T lub NEO-6M/7M)

Autor: Jarosław Marek Niewiński (jmnlabs) **Asystenci:** Claude Opus 5 (Anthropic) · GLM-5.3 Max (Z.ai) · Qwen3.8-Max — polska i hiszpańska wersja tego podręcznika jest autorstwa GLM-5.3 Max Ta instrukcja nie zakłada **niczego**. Jeśli nigdy nie kompilowałeś firmware, nie wgrywałeś go do mikrokontrolera i nie używałeś terminala szeregowego — zacznij od Części 1 i rób dokładnie to, co tam napisano, po kolei. Każdy krok mówi, co wpisać i co powinieneś zobaczyć. Nic ważnego nie zostawiono jako „zadanie dla czytelnika".

> **Streszczenie w 60 sekund:** zainstaluj Arduino IDE z rdzeniem STM32 (nie 3.0.0), otwórz szkic, wybierz swój wyświetlacz w `gpsdo_config.h`, kompiluj z *Newlib Nano + Float printf*, wgraj przez ST-Link lub DFU. **Nigdy nie rób pełnego wymazania chipu (Erase Chip)** — twoje ustawienia i kalibracja żyją w sektorze 7 i tylko pełne wymazanie może je zniszczyć. Potem, po serialu na 115200: `CT`, poczekaj, `LC`, poczekaj, `LA 12`, `SAW 1`, `ES`. Patrz, jak faza trzyma się zera. Gotowe.

Spis treści

- Część 1 — Kompilacja firmware
- Część 2 — Wgrywanie i zasada sektora 7
- Część 3 — Pierwsze uruchomienie: kalibracja, algorytm, zapis
- Część 4 — Czternaście algorytmów (0–13) i ich parametry
- Część 5 — Wyświetlacze: co znaczy każde pole
- Część 6 — Telemetria serialowa (raport 6-liniowy)
- Część 7 — Spis komend (wszystkie)
- Część 8 — Tuner na PC
- Część 9 — Ustawienia, flash ring i zużycie
- Część 10 — Diagnostyka problemów
- Aneks A — Jak działa GPSDO, prostymi słowami
- Aneks B — PID dla opornych
- Aneks C — Słowniczek
- Aneks D — Filtr Kalmana prostymi słowami

Część 1 — Kompilacja firmware

1.1 Co trzeba zainstalować

1. **Arduino IDE** (1.8.x lub 2.x — oba działają).
2. **Rdzeń STM32duino w wersji 2.2.0 do 2.12.0.** Boards Manager → szukaj „stm32" → **STM32 MCU based boards by STMicroelectronics**. Zainstaluj 2.12.0 (ostatnia wspierana).

> **UWAGA — rdzeń 3.0.0 nie działa.** Wyszedł w lipcu 2026 i łamie ten projekt na trzy sposoby: `ltoa()` już nie istnieje (błąd kompilacji), `HardwareSerial Serial2(PA3, PA2)` się nie kompiluje, a TFT_eSPI przestaje sterować panelami (trwały biały ekran). Trzymaj 2.12.0 lub starszy, dopóki projekt nie ogłosi inaczej.

1. **Biblioteki** (Library Manager, instaluj po dokładnej nazwie):

Biblioteka	Po co
STM32duino FreeRTOS	system operacyjny
TinyGPS++	parsowanie zdań GPS
U8g2	wyświetlacze OLED (tylko gdy jakiś włączysz)
Adafruit AHTX0	czujnik temperatury/wilgotności AHT10/AHT20
Adafruit BMP280	czujnik ciśnienia/temperatury BMP280
Adafruit INA219	monitor napięcia/prądu zasilania
hd44780	LCD znakowy 20x4 (tylko gdy włączysz)
TFT_eSPI	wyświetlacz TFT na SPI

1. Nic więcej. Nie ma biblioteki EEPROM — ustawienia żyją we flashu wbudowanym (Część 2).

1.2 Otwarcie szkicu

Rozpakuj projekt. Otwórz `GPSDO_FreeRTOS/GPSDO_FreeRTOS.ino` w Arduino IDE. Folder musi zachować nazwę — Arduino wymaga, żeby nazwa folderu zgadzała się z plikiem `.ino`.

1.3 Dobór sprzętu w `gpsdo_config.h`

Ten jeden plik decyduje, co się skompiluje. Linie aktywne zaczynają się od `#define`; wyłączone od `//`. Odkomentuj dokładnie to, co masz na płytce, zakomentuj to, czego nie masz. Konfiguracja fabryczna to w pełni obsadzona płytka (duży TFT + wszystkie czujniki); dopasuj ją do rzeczywistości.

Wyświetlacz główny — wybierz dokładnie jeden:

Flaga	Sprzęt
`GPSDO_TFT_ILI9488`	SPI TFT 320x480 (domyślny; pracuje w poziomie 480x320)
`GPSDO_TFT_ST7789`	SPI TFT 240x320
`GPSDO_TFT_ILI9341`	SPI TFT 240x320
`GPSDO_OLED_SH1106` / `_SSD1306` / `_SSD1309`	OLED 128x64 na I2C (tylko jeden)
`GPSDO_LCD_20x4`	HD44780 20x4 przez PCF8574 na I2C
`GPSDO_TM1637` / `GPSDO_TM1637_6`	zegar LED 4-cyfrowy / 6-cyfrowy (wzajemnie wyłączne, i nie razem z LCD)
`GPSDO_HT16K33`	zegar LED 4-cyfrowy na I2C (domyślnie włączony; niezależny od wyświetlacza głównego)

Czujniki i dodatki (fabrycznie włączone — zakomentuj, czego nie masz):

Flaga	Sprzęt
`GPSDO_AHT10`	AHT10/AHT20 na I2C
`GPSDO_BMP280_I2C`	BMP280 na I2C
`GPSDO_INA219`	INA219 na I2C
`GPSDO_LM75`	termometr z rodziny LM75 na I2C — LM75A/B, PCT2075, TMP75/175, TMP1075 — pod `GPSDO_LM75_ADDR` (0x48); bez biblioteki. Jego temperatura idzie po `CPU:` w linii czujników raportu `RH` (build 68). Inny układ pod 0x48 może udawać termometr (ADS1x15, PCF8591): zmień wtedy adres albo wyłącz tę flagę
`GPSDO_VCC` / `GPSDO_VDD`	pomiar szyny 5 V / 3,3 V
`GPSDO_UBX_CONFIG`	ustawienie NEO-6M/7M w tryb binarny przy starcie
`GPSDO_FAKE_UBLOX`	chiński **klon** u-bloxa: tylko proba baudrate, zero konfiguracji UBX (patrz Część 3.2)
`GPSDO_GPS_TIMING`	obsługa odbiorników timingowych (LEA-6T/M8T/NEO-M8T/ZED-F9T): survey-in, qErr
`GPSDO_PICDIV`	wyjście zbrojenia dzielnika picDIV na PB3
`GPSDO_LTIC`	**sprzętowy detektor fazy** (ramp TIC na PA1) — wymagany dla algorytmów 10, 11, 12. Włączaj TYLKO gdy detektor jest fizycznie zamontowany: bez niego PA1 pływa, a pętla dyscyplinuje OCXO szumem ADC
`GPSDO_LTIC_ACTIVE_RESET`	rozładowanie kondensatora detektora przez PA1 zaraz po każdym odczycie (domyślnie włączone; zostaw włączone przy obu detektorach — klasycznym RC Kaashoeka i wariancie ze źródłem prądowym — patrz Część 3.3)
`GPSDO_PWM_DITHER`	24-bitowe napięcie sterujące z ditherowanego PWM 13-bit (nie wyłączaj — to ten dobry DAC)
`GPSDO_DAC_EXT`	zewnętrzny DAC AD5680, 18-bit, bit-bang na PB4/PB0/PB2 — można kompilować razem z `GPSDO_PWM_DITHER`; żywą ścieżkę wybiera komenda `DAC` w czasie działania. Po cichu wyłącza `GPSDO_TM1637`/`GPSDO_TM1637_6` i `GPSDO_GEN_2kHz_PB5` (PB4 to chip-select DAC-a)
`GPSDO_SPAN_SENSE`	drugi biegun zworki zakresu EFC na **PB14** (dostarczane WŁĄCZONE): jedna kalibracja `CT` na każdą pozycję zakresu, przełączana, gdy zworka się przestawi — zob. `SPAN` w Części 7. Płytka bez tego przewodu czyta FULL na wewnętrznym podciągnięciu i zachowuje swoją jedną kalibrację. Wyłącz tylko wtedy, gdy PB14 jest na Twojej płytce podłączony do czegoś innego
`GPSDO_GEN_2kHz_PB5`	prostokąt testowy 2 kHz na PB5
`GPSDO_BLUETOOTH`	moduł HC-06 na PA2/PA3
`GPSDO_BLUETOOTH_PARALLEL`	USB **i** Bluetooth równocześnie (domyślnie włączony)

Plik odmawia kompilacji (celowo), jeśli wybierzesz dwa wyświetlacze tego samego typu albo dwa elementy kłócące się o te same piny. Przeczytaj komunikat `#error` — nazywa konflikt.

Zostaw włączone `GPSDO_LTIC` i `GPSDO_PWM_DITHER`, chyba że naprawdę nie masz sprzętu detektora: bez LTIC tracisz algorytmy 10–13, czyli cały sens v1.08.

1.4 Konfiguracja TFT_eSPI (tylko TFT)

TFT_eSPI konfiguruje się **w bibliotece**, nie w szkicu. Znajdź `Arduino/libraries/TFT_eSPI/User_Setup.h` i umieść w nim blok dla swojego panelu:

240x320 (ST7789 lub ILI9341):

```
#define ST7789_DRIVER          // or ILI9341_DRIVER
#define TFT_WIDTH  240
#define TFT_HEIGHT 320
#define TFT_MISO PA6      // required on STM32 even if the display has no MISO pin
#define TFT_MOSI PA7
#define TFT_SCLK PA5
#define TFT_CS   PB13
#define TFT_DC   PB12
#define TFT_RST  PB15
#define TFT_RGB_ORDER TFT_BGR
#define TFT_INVERSION_OFF       // fixes inverted colours on some ST7789 modules
#define LOAD_GLCD               // font 1 — frequency readout + splash
#define LOAD_FONT2              // font 2 — header + data grid
#define LOAD_FONT4              // font 4 — status bar, busy messages
#define SPI_FREQUENCY 40000000  // 40 MHz works; drop to 27 MHz for long wires
```

320x480 (ILI9488 / ILI9486):

```
#define ILI9488_DRIVER          // works for both ILI9488 and ILI9486
#define TFT_WIDTH  320
#define TFT_HEIGHT 480
// same TFT_MISO/MOSI/SCLK/CS/DC/RST/RGB_ORDER lines as above
#define LOAD_GLCD               // font 1 — splash credits
#define LOAD_FONT4              // font 4 — splash subtitle path
#define LOAD_GFXFF              // REQUIRED on this panel — free fonts
#define SPI_FREQUENCY 40000000  // don't skimp: this panel pushes 2.4x the pixels
```

Podłączenie panelu: SCK→PA5, SDI→PA7, RES→PB15, D/C→PB12, CS→PB13. `TFT_MISO PA6` musi być zdefiniowane, nawet jeśli nic do niego nie jest podłączone.

1.5 `build_opt.h` — zostaw w spokoju

Plik zawiera trzy flagi kompilatora i nie wymaga od Ciebie żadnej akcji:

```
-DSERIAL_RX_BUFFER_SIZE=256 -DSERIAL_TX_BUFFER_SIZE=512
-DCDC_TRANSMIT_QUEUE_BUFFER_PACKET_NUMBER=16
```

Pierwsza linia powiększa bufory seriala, żeby zdania GPS nie ginęły. Druga rozszerza kolejkę nadawczą USB-CDC ze 128 bajtów do 1 KB, dzięki czemu raport telemetrii 1 Hz (około 400 znaków) zawsze się mieści — bez tego host, który podłączy się i nie czyta, mógłby wstrzymać raport na sekundy. Plik jest podbierany automatycznie przez rdzeń STM32. Nie usuwaj go.

1.6 Menu Tools — płytka i biblioteka uruchomieniowa

W Arduino IDE: **Tools** →

- **Board:** "Generic STM32F4 series" → **BlackPill F411CE**.
- **C Runtime Library: Newlib Nano + Float Printf/Scanf**. To obowiązkowe. Bez tego firmware się skompiluje, ale wypisze śmieci tam, gdzie powinna być liczba zmiennoprzecinkowa (wszystkie napięcia, częstotliwości i fazy).

1.7 Włącz USB CDC (żeby płytka dała konsolę po USB)

CDC sprawia, że `Serial` staje się wirtualnym portem COM na kablu USB. Banner startowy, wiersz poleceń i raport 1 Hz — wszystko tym jedzie. Bez CDC płytka nadal działa — wyświetlacz się odświeża, pętla dyscyplinuje oscylator — ale po USB wygląda na całkiem martwą: brak banera, brak znaku zachęty, nic.

1. **Tools → USB support (lub "USB support (if available)") → "CDC (generic Serial supersedes U(S)ART)".** Dokładna nazwa różni się nieco między wersjami rdzenia; wybierz tę, która wspomina **CDC** i **generic Serial**.
2. Skompiluj ponownie i wgraj. Zmiana tego ustawienia zmienia firmware — to nie jest przełącznik działający w locie.
3. Po wgraniu **naciśnij raz przycisk RESET na płytce**. Chip przełącza się z urządzenia USB DFU (wgrywanie) na urządzenie CDC serial; bez resetu część hostów rozmawia dalej ze starym, już nieistniejącym urządzeniem.
4. Pojawia się nowy port COM (Windows: "STM32 Virtual COM Port", **VID 0483, PID 5740**). Jeśli Windows pokazuje "Unknown device" albo "device descriptor request failed", zainstaluj raz sterownik przez Zadig — patrz też Część 2.5.
5. Otwórz port na 115200. Firmware **czeka do 3 sekund** po resecie, aż host otworzy port, zanim wydrukuje banner — wolny sterownik nie połka pierwszych linii. Jeśli otwierasz terminal i nic nie widzisz, naciśnij RESET jeszcze raz przy już otwartym terminalu.

Dwie rzeczy warte zapamiętania:

- Tryb DFU (wgrywanie) i tryb CDC (praca) to dla Windows **dwa różne urządzenia USB**. Pierwsze cykle wgrywania mogą każdy raz odpalić rundę instalacji sterownika — to normalne, po chwili się uspokaja.
- Przy skompilowanym `GPSDO_BLUETOOTH_PARALLEL` (domyślna konfiguracja) wszystko, co widać po USB, jest dublowane na UART Bluetooth przy **57600** — gotowa zapasowa konsola, gdyby USB kiedyś stawiało opór. Raport co sekundę idzie do każdego portu osobno (build 68), więc gdy host USB jest podłączony, ale nie czyta, traci się co najwyżej kopię raportu na USB — nigdy tę na Bluetooth — i nigdy nie zatrzymuje to płytki.

1.8 Kompilacja i kontrola rozmiaru

Sketch → Verify/Compile. Po zakończeniu przeczytaj linię `Sketch uses NNNNNN bytes`.

> **NNNNNN musi zostać poniżej 393216.** To 384 KB — wszystko powyżej należy do pierścienia ustawień w sektorze 7 (Część 2). Zignoruj procent, który wypisuje IDE: liczony jest względem pełnych 512 KB chipu, więc build, który „mieści się" procentowo, może przekroczyć granicę sektora 7. Samo v1.08 zajmuje około 300 KB (build 71: 303 364 bajty) — zapas duży, ale nie ignoruj nagłego wzrostu.
>
> **Który wsad naprawdę działa?** Baner wypisuje czas kompilacji — a ten czas należy do **szkicu**, nie do firmware'u. Builder Arduino nie przekompilowuje jednostki, której źródła się nie zmieniły, więc zmiana `GPSDO_algorithms.cpp` i wgranie zostawia plik obiektowy szkicu nietknięty, ze starszą datą w środku. Baner wypisuje też `image CRC32`, liczone przy starcie z samej pamięci: nie wyjdzie z cache'a buildera, zmienia się przy zmianie dowolnego bajtu dowolnej jednostki kompilacji, a dwie płytki z tym samym wsadem pokażą tę samą liczbę. Gdy log i pamięć się nie zgadzają, wierz tej liczbie. `V` pokazuje ją w każdej chwili.
>
> Jeśli stempel czasu też ma być prawdziwy, uruchom przed kompilacją `python3 tools/bumpbuild.py` — dotyka `build_id.h`, który szkic dołącza wyłącznie w tym celu, a to zmusza builder do przekompilowania szkicu i tylko szkicu. Numer pojawia się w banerze jako `build N`.

Część 2 — Wgrywanie i zasada sektora 7

2.1 Jak podzielony jest flash 512 KB

Sektory	Zakres adresów	Zawartość
0 – 5	0x08000000 – 0x0803FFFF	**Twój firmware** (384 KB)
6	0x08040000 – 0x0805FFFF	zapas (bufor wzrostu)
7	**0x08060000 – 0x0807FFFF**	**flash ring: ustawienia + kalibracja + dane wyuczone**

W sektorze 7 mieszka wszystko, czego urządzenie nauczyło się o samym sobie:

- twoje zapisane ustawienia (wybór algorytmu, wszystkie parametry pętli, strefa czasowa),
- kalibrację czułości oscylatora z `CT`,
- kalibrację detektora fazy z `LC`,
- wyuczony model dryfu/tłumienia i ostatni punkt pracy.

Odtworzenie tego wszystkiego kosztuje godzinę na stole. Ochrona sektora 7 jest więc **najważniejszą zasadą eksploatacyjną całego projektu**.

2.2 Złota zasada

> **Zwykłe wgrywanie nigdy nie dotyka sektora 7. Pełne wymazanie chipu go niszczy.**
>
> - Upload z Arduino IDE, DFU i bootloader STM32duino przeprogramowują tylko sektory 0–5. Ustawienia przeżywają każdą rutynową aktualizację firmware.
> - „Erase Chip" w ST-LINK Utility / STM32CubeProgrammer albo `erase` bez zakresu adresów w J-Link wymazuje cały chip **razem z sektorem 7**. Nigdy nie używaj tego na działającej, skalibrowanej jednostce.

Jeśli wgrywasz ręcznie przez ST-Link lub J-Link, wymazuj **wyłącznie** zakres firmware i dopiero wtedy ładuj:

```
> erase 0x08000000 0x0803FFFF
> loadbin firmware.bin 0x08000000
```

Jeśli sektor 7 kiedyś zginie: nic się nie psuje. Przy następnym starcie firmware wykrywa pusty pierścień, formatuje go i startuje z wartościami domyślnymi — po prostu ponawiasz `CT`, `LC`, ustawiasz algorytm i strefę czasową, i `ES`. Tracisz strojenie, nie urządzenie.

Przed pierwszym wgrywaniem na skończoną jednostkę zrób pełny backup (przykład J-Link; ST-LINK Utility ma odpowiedni przycisk "save"):

```
> JLinkExe -device STM32F411CE -if SWD -speed 4000
> savebin backup_full.bin 0x08000000 0x80000
> exit
```

Przywrócenie: `loadbin backup_full.bin 0x08000000`.

2.3 Wgrywanie metodą A — ST-Link (najprościej)

1. Podłącz ST-Link V2 do pinów SWD BlackPilla (SWDIO, SWCLK, GND, 3V3).
2. Arduino IDE → Tools → **Upload method: "STM32CubeProgrammer (SWD)"**.
3. Naciśnij Upload. Gotowe. Programator zapisuje tylko to, co zajmuje szkic.

2.4 Wgrywanie metodą B — DFU po USB (bez programatora)

> **Jaki BOOT0 masz na płytce?** Starsze BlackPille WeAct mają zworkę BOOT0; aktualne (v3.1) mają tylko **przycisk BOOT0**. Użyj przepisu dla swojego wariantu.

Płytka ze zworką: przełóż zworkę BOOT0 na **1**, naciśnij RESET — płytka pojawi się jako "STM32 BOOTLOADER". Po uploadie wróć zworką do **0** i naciśnij RESET.

Płytka z przyciskiem (v3.1), niezawodnie: odłącz zasilanie zewnętrzne; **naciśnij i trzymaj BOOT0**, podłącz kabel USB **trzymając przycisk**; po kilku sekundach puść BOOT0 — Windows nie powinien zgłosić "nieznane urządzenie", a STM32CubeProgrammer widzi płytkę. Wgraj, potem naciśnij RESET. (Trzymanie BOOT0 z jednoczesnym stukaniem NRST, jak podają inne źródła w internecie, zwykle **nie** działa — zwłaszcza gdy płytka jest już wlutowana w PCB.)

Windows może pierwszy raz poprosić o sterownik — patrz uwaga niżej.

2.5 Po każdym wgrywaniu: uwaga o USB

- Po zakończeniu uploadu **naciśnij przycisk RESET na płytce** — urządzenie USB czysto przeenumeruje się z trybu bootloadera na CDC serial firmware.
- USB serial firmware to **VID 0483, PID 5740** ("STM32 Virtual COM Port"). Jeśli Windows odmawia otwarcia portu, zainstaluj raz sterownik przez Zadig.

Część 3 — Pierwsze uruchomienie: kalibracja, algorytm, zapis

Potrzebny jest program terminala (PuTTY, Tera Term, monitor serialowy Arduino IDE albo tuner z Części 8 — tuner jest najwygodniejszy).

3.1 Połączenie

1. Podłącz USB (i/lub Bluetooth na 57600 — w konfiguracji fabrycznej oba działają równocześnie).
2. Otwórz port na **115200, 8N1**. Końce linii: CR lub LF — oba działają.
3. Naciśnij **Enter**. Powinieneś zobaczyć znak zachęty albo start raportu 1 Hz.
4. Wpisz `H` i Enter. Przewinie się pełna lista komend — to Część 7 tej instrukcji, na żywo.

Log startowy linia po linii. Pierwsze dziesięć sekund po resecie wypisuje stałą sekwencję. Oto typowa z działającej, już skonfigurowanej jednostki (dokładne linie zależą od sprzętu), a dalej — co każda znaczy:

```
================================================
GPSDO v1.08.71rt compiled 2026-10-29 23:34:12  build 71
image CRC32 8EED4610  (303364 bytes of flash)
FreeRTOS port by J. M. Niewinski  with Claude, GLM-5.3 Max & Qwen3.8-Max AI
https://github.com/jmnlabs/GPSDO_FreeRTOS
Inspired by GPSDO v0.06c by Andre Balsa
https://github.com/AndrewBCN/STM32-GPSDO
Algos 0-2 original design by Andre Balsa
Algos 3-9 by J. M. Niewinski
Algo 10 (LTIC 3-stage) inspired by Dan Wiering's measurements
Algo 11 (LTIC-Lars) after Lars Walenius' PI loop
Algo 12 (multi-level accumulator) after Alan Cashin (MIS42N)
Algo 13 (Kalman filter) by J. M. Niewinski - original to this project
Type H = help  SW = stack diagnostics
================================================
Reset cause: POWER-ON/BROWN-OUT PIN/NRST
Flash ring: sector 7 ready
Settings: recalled from flash ring
Live store: LRN + LC applied from flash ring
Initial PWM=44653 algo=12 time_offset_min=120
GPS init: probing baud rate...
GPS detected at 38400
GPS: disabling noisy NMEA at 38400 baud
GPS: 4/4 NMEA sentences disabled
GPS: sending UBX config at 38400 baud
UBX: CFG-NAV5 ACK
LEA-T: accepted CFG-TMODE2 (28B)
Hardware configured, creating RTOS objects...
Timers started
Starting FreeRTOS scheduler
HW: AHT10/AHT20 sensor    OK  (I2C 0x38)
HW: BMP280 sensor         OK  (I2C 0x77)
HW: INA219 sensor         OK  (I2C 0x40)
HW: LM75 sensor           OK  (I2C 0x48)
HW: LTIC phase input      enabled (PA1) - needs the ramp detector hw
TFT: init start (SPI1 PA5/PA7, CS=PB13 DC=PB12 RST=PB15)
TFT: freq-band sprite (4-bit) created
TFT: header sprite (4-bit) created
TFT: data sprite (1-bit) created
```

Potem startuje raport 1 Hz (Część 6). Co mówi każda linia:

Linia	Znaczenie
banner (`GPSDO v1.08.71rt …`)	tożsamość firmware — wydanie v1.08, build 71 (`.71rt`). Jeśli zamiast tego widzisz śmieci: zła prędkość — użyj 115200
`image CRC32 …`	suma kontrolna programu we flashu, liczona przy starcie: wskazuje binarkę, która naprawdę działa, nawet gdy czas kompilacji wyżej jest nieaktualny (zob. 1.8)

Linia	Znaczenie
`Reset cause:`	dlaczego chip się zrestartował. `POWER-ON/BROWN-OUT PIN/NRST` = normalne włączenie zasilania lub przycisk reset. `SOFTWARE` = reset zarządzony przez firmware (`RB`, koniec uploadu). `INDEP-WDG/WINDOW-WDG` = watchdog (ten firmware żadnego nie używa — traktuj jako sygnał awarii). Dodatkowa linia `-> supply dipped: check the 3V3 rail under load` pojawia się po brown-out: szyna 3V3 siadła pod obciążeniem OCXO — napraw zasilanie, nie ignoruj tego
`Flash ring: sector 7 ready`	pierścień ustawień w sektorze 7 znaleziony i poprawny
`Flash ring: sector 7 blank/formatted (defaults)`	pusty lub wymazany pierścień — **normalne przy pierwszym wgrywaniu**; firmware go formatuje i używa wartości domyślnych z kompilacji
`Settings: recalled from flash ring`	zapisane ustawienia (algorytm, parametry pętli, strefa czasowa, flagi) zostały zastosowane
`Settings: none stored (compile-time defaults)`	nic jeszcze nie zapisano — oczekiwane na nowej jednostce; przejdź Część 3 i wykonaj `ES`
`Live store: LRN + LC applied from flash ring`	dane wyuczone (kalibracja LC, model dryfu, ostatni punkt pracy) zastosowane — linia pojawia się dopiero na jednostce kalibrowanej
`Initial PWM=… algo=… time_offset_min=…`	punkt pracy wybrany przy starcie: ostateczny kod PWM (dane wyuczone nadpisują ustawienia, gdy są świeższe), przywrócony algorytm i przesunięcie strefy czasowej w minutach
`GPS init: probing baud rate...` / `GPS detected at …`	odbiornik znaleziony, prędkość zmierzona (preferuje 38400)
`GPS: no response to baud probe, defaulting to 9600`	**żaden odbiornik nie odpowiedział** — zanim ruszysz dalej, sprawdź okablowanie modułu GPS
`GPS: disabling noisy NMEA…` / `GPS: sending UBX config…` / `UBX: CFG-NAV5 ACK`	odbiornik przechodzi w tryb binarny i dynamikę stacjonarną; warianty `NAK`/`no response` są przeżywalne — odbiornik po prostu zachowuje obecną konfigurację
`LEA-T: accepted CFG-TMODE2`	odbiornik timingowy (klasa LEA-M8T) skonfigurowany w survey-in / Time Mode — pojawia się tylko przy `GPSDO_GPS_TIMING` i odbiorniku timingowym
`Hardware configured…` / `Timers started` / `Starting FreeRTOS scheduler`	wewnętrzny start; wszystkie trzy zawsze powinny się pojawić
`HW: <czujnik> OK (I2C …)` / `HW: <czujnik> not found`	skan magistrali I2C: każdy czujnik raportuje obecność lub jej brak. Brakujący czujnik nie jest fatalny — jednostka działa bez niego, tylko pole w raportach zostaje puste
`HW: LTIC phase input enabled (PA1)`	firmware ZBUDOWANO ze ścieżką detektora fazy, a PA1 jest pod nią skonfigurowane — nie odróżnia zamontowanego detektora od pływającego pinu, więc to nie jest test sprzętu (algorytmy 10–13 zależą od prawdziwego detektora)
`TFT: init start (…)`	start inicjalizacji wyświetlacza. **Jeśli po tej linii nic nie następuje**, okablowanie TFT albo `User_Setup.h` jest złe (Część 1.4) — to przypadek „ekran biały/martwy, ale serial żyje"
`TFT: … sprite FAILED — direct-draw fallback`	wyświetlacz działa, ale wolniej się przerysowuje (zabrakło RAM na sprity anty-miganiowe) — kosmetyka, nie awaria

Linie pojawiające się **później**, w trakcie pracy, i ich znaczenie:

Linia	Znaczenie
`LTIC: running UNCALIBRATED (run LC)`	aktywny jest algorytm 10/11/12, ale `LC` nigdy nie było uruchamiane — liczby fazy są nominalne, nie zmierzone. Uruchom `LC`
`LTIC ACQ: polarity unset — run 'LPOL -1' (or +1)`	algorytm 10 odmawia sterowania do ustawienia polaryzacji (bezpiecznie czeka zamiast zgadywać). Algorytmy 11 i 12 zakładały **+1** i sterowały mimo to; teraz czekają tak samo
`picDIV: armed (output stopped, waiting for 1PPS sync)`	dzielnik zbrojony; przerwa 1–1,2 s na jego wyjściu jest oczekiwana, zanim zsynchronizuje się do kolejnego zbocza PPS
`picDIV: no GPS fix or no 1PPS - arming deferred until both are back`	o uzbrojenie poproszono bez fixa albo bez impulsu w ostatnich 1,5 s (build 68): prośba czeka i zostaje wykonana, gdy oba wrócą
`GPS 1PPS lost — auto-holdover engaged` / `GPS fix lost — auto-holdover engaged`	odniesienie znikło — 10 s bez impulsu albo utrata fixa — i pętla sama się zamroziła (A.5); `GPS fix recovered — auto-holdover disengaged` pojawia się, gdy fix wróci razem z 10 s nieprzerwanych impulsów. Pod `MH` linia kończy się zamiast tego słowami `manual holdover kept`, a holdover zostaje Twój (build 68)
`LEA-T: … survey … %/s) — continuing anyway`	monitor survey-in; „continuing anyway" znaczy, że survey osiągnął limit czasu powyżej celu dokładności — zwykle zaczepiony widok nieba
`!!! FreeRTOS …` (configASSERT / STACK OVERFLOW / MALLOC FAILED)	firmware złapał awarię, z plikiem/linią albo nazwą taska — niebieska LED miga w tym samym czasie szybko. Zanotuj jedno i drugie; jednostka potrzebuje resetu (`RB`)

Dziewicza jednostka automatycznie zarządza kalibrację przy pierwszym starcie — możesz zobaczyć odliczanie kalibracji zanim cokolwiek innego.

3.2 Pozwól GPS-owi się ustabilizować

- Daj antenie czysty widok nieba. Parapet działa; geodezyjna antena na dachu działa lepiej.
- Przy odbiorniku timingowym (M8T/F9T...) i skompilowanym `GPSDO_GPS_TIMING` przy starcie działa **survey-in**: odbiornik mierzy własną pozycję przez co najmniej 300 s, dopóki szacunek nie będzie lepszy niż 5 m, po czym przechodzi w **Time Mode** na stałej pozycji. Od tej chwili wyświetlacz pokazuje `HDOP:TIME` zamiast liczby. Survey-in musi się zakończyć tylko raz; powtarza się po zaniku zasilania. (`SV 0` wyłącza, `SV 1` włącza z powrotem, przy następnym starcie.)

Jak zapisać survey na stałe. Survey 300 s / 5 m, który firmware robi przy starcie, jest kompromisem: dość długi, by się przydał, dość krótki, by nikt na niego nie czekał. Jeśli możesz zostawić odbiornik na kilka godzin, zrób survey raz w u-center i zapisz go w podtrzymywanej bateryjnie konfiguracji modułu — u-center **V8.29** ma właściwe komendy (UBX-CFG-TMODE2 uruchamia survey, potem UBX-CFG-CFG → *Save current configuration*). Długi survey daje lepszą pozycję stałą i przeżywa wyłączenia zasilania: firmware sprawdza Time Mode przy starcie i pomija własny survey, gdy odbiornik już go zgłasza. To zalecenie Alana Cashina — najtańsza dokładność w całej konstrukcji. - Rozgrzewka OCXO trwa 300 s (`WU 0` ją pomija; zostaw włączoną). - Żółta LED jest **wyłączona bez fixa GPS, świeci ciągle przy fixie** — gdy coś wygląda źle, zaglądaj tam najpierw.

Jaki moduł GNSS wybrać? Oryginalny odbiornik timingowy u-blox (klasa LEA-M8T) to baza, pod którą to firmware zbudowano: survey-in, Time Mode, korekta piły `qErr`. Dyscyplinowanie zadziała też na tanich modułach nawigacyjnych — w tym na chińskich klonach u-bloxa z eBay/AliExpress — bo pętli wystarczy impuls 1PPS i zdania NMEA. Ale klony ignorują konfigurację binarną: spodziewaj się `0/4 NMEA sentences disabled`, braku ACK dla ramek CFG i ~15 s dłuższego startu, póki nie wygasną timeouty; nie ma survey-in ani `qErr`, wyświetlacz pokazuje stale liczbowe HDOP, a dodatkowa wędrówka pozycji wbija się w fazę — i to właśnie wchłaniają luźne domyślne limity algo-12. Znany dziwaczek klonów: po nieudanej rundzie konfiguracji niektóre moduły przestają wysyłać dane i wyświetlacz stoi na "acquiring" — lekiem jest wejście w tunel u-center (`T` z baudem modułu, np. `T 9600`) i zwykłe poczekanie do wygaśnięcia: ponowna proba po tunelu przywraca pracę. Zajiście tego wszystkiego jest jednym przełącznikiem: zdefiniuj **`GPSDO_FAKE_UBLOX`** w `gpsdo_config.h`, a firmware zrobi tylko probę baudrate i nic więcej nie wyśle — niezależnie od

innych włączonych opcji GPS. Jeszcze nie testowany na sprzęcie (autor nie ma modułu-klonu); raporty z pola mile widziane.

3.3 Kalibracja — najpierw `CT`, potem `LC`. Kolejność ma znaczenie.

Te dwie komendy uczą firmware dwóch różnych faktów fizycznych, a druga potrzebuje pierwszego:

Krok 1 — `CT` (czułość oscylatora + strojenie pętli, ~3 minuty; ~11 przy łagodnym obiekcie). Wpisz `CT` i Enter. Firmware prowadzi napięcie sterujące przez trzy punkty (1,5 V; 2,0 V; 2,5 V), mierzy częstotliwość w każdym, dopasowuje prostą i liczy **K — ile herców jest wart jeden krok PWM na *Twoim* oscylatorze** (akceptowany zakres 0,01–2 mHz/LSB). Z K wyprowadza sensowne wzmocnienia PID dla algorytmów 3–9 i pętli LTIC oraz **sam zapisuje grupę PID** do flasha. Przy łagodnym obiekcie — zawężony zakres EFC albo zewnętrzny DAC, K poniżej 0,12 mHz/LSB — drugi przebieg mierzy ponownie wokół punktu 10 MHz na średnich 100-sekundowych, co dodaje około ośmiu minut. Nie odcinaj zasilania, dopóki trwa. Na płytce ze zworką zakresu (PB14) uruchom je w pozycji, której używasz; druga pozycja dostanie własne `CT` sama, gdy zworka pierwszy raz tam trafi.

Krok 2 — `LC` (kalibracja detektora fazy, ~5 minut). Wpisz `LC` i Enter. Firmware zbroi dzielnik picDIV, centruje detektor, przeprowadza jedną rampę kondensatora i mierzy **ns na wolt**, **przesunięcie zera w woltach** i **zakres w ns** detektora. Przy PASS **sam się zapisuje**. Odmawia użyteczności, jeśli `CT` nie było uruchomione — dlatego kolejność ma znaczenie. Gdy dzielnik położy fazę poniżej pasma detektora, `LC` czeka, aż własne przemiatanie wprowadzi ją do pasma (build 68: najwyżej dwie minuty, `LC: ramp wait … s` co 10 s, drugie uzbrojenie, gdy to nie wystarczy), więc na takiej płytce trwa trochę dłużej. Uzbraja tylko przy fixie GPS i impulsie 1PPS, do którego da się zsynchronizować dzielnik: bez nich kończy się komunikatem `LC: ERROR no GPS fix or no 1PPS to sync the picDIV to — aborting.` i przywraca kod pętli — sprawdź antenę i uruchom je ponownie.

Kontrola: wpisz `LL` i sprawdź, że blok LTIC ma niezerowe `ns_per_volt`, `zero_offset`, `range_ns`. Wpisz `DAC` — powie Ci nawet, ile mikroherców wart jest jeden krok wyjścia.

Jak powstaje odczyt (build 62). Tuż przed każdym zboczem PPS firmware wstrzymuje na układzie wszystko inne — także przerwania, na kilkanaście mikrosekund wokół zbocza — i od 5 µs po zboczu robi 64 pomiary kondensatora bezpośrednio na rejestrach przetwornika, łącznie ok. 110 µs; prowadzi przez nie prostą względem ich czasów z TIM2 i odczytuje ją na samym zboczu, zanim cokolwiek z ładunku upłynie. `LB` pokazuje ostatnią serię: „taken at the edge", 64 malejące kody, prostą, rozrzut wokół niej (2–3 kody rms to norma), tempo spadku jako stałą czasową (ok. 1,4 ms — to głównie sam przetwornik: każdy pomiar zabiera ok. 0,12 % ładunku, ~3 kody; sam upływ jest dużo wolniejszy) oraz wygięcie obok typowego dla płytki (na płytce autora ok. −3 kody). Każda seria jest trochę wygięta, co sekundę tak samo; odrzucana jest tylko taka, która wygina się o 3,5 kodu lub więcej inaczej niż pozostałe. Pomiar odstający od prostej o więcej niż sześć kodów (i cztery MAD) jest pomijany, a prosta liczona od nowa — od buildu 70 do dwudziestu razy (od buildu 67 do dwunastu, wcześniej osiem): detektor, który łapie falę prostokątną, dostaje parę zakłóceń na każdym jej zboczu — na płytce Dana cztery–pięć par na serię, co ok. 21 µs, od czegoś poza GPSDO, co przełącza z częstotliwością ok. 23 kHz (zostają przy pinie PWM trzymanym nisko i pojawiają się i znikają zależnie od pory dnia) — i przy dwunastu seria z sześcioma parami lub więcej była odrzucana. Wiele serii odrzuconych w `LB` jako „off the line" wskazuje na takie zakłócenie na detektorze. Od buildu 71 seria, w której co trzeci pomiar odstaje razem, traci najpierw całą tę trzecią część, zanim odpadnie którykolwiek pojedynczy pomiar: w części serii pomiary następują po sobie co 16⅔ okresu 10 MHz (w innych co ~17,7), więc co trzeci trafia w ten sam punkt jego okresu, a detektor, który łapie jego zbocza, ma jedną trzecią serii przesuniętą o tych samych kilka kodów — na płytce Dana o 17–30 kodów w górę albo o 7,5 w dół. Taka trzecia część odsuwała prostą o kilka kodów albo ponad dwadzieścia pomiarów wypadało z prostej jeden po drugim i seria była odrzucana. Jest pomijana, gdy odstaje od dwóch pozostałych o więcej niż 6 kodów i trzykrotność własnego szumu serii, te dwie się zgadzają, a dwie trzecie jej pomiarów leżą co najmniej w połowie tej odległości; dwadzieścia pominięć zostaje na zakłócenia. `LB` pisze wtedy `every third conversion from #0 left out: +29.6 codes off the other two`, a w przeciwnym razie, jak daleko jest najdalsza trzecia część (`thirds kept: the furthest -0.5 codes off the other two`). Typowego wygięcia płytki firmware uczy się wyłącznie z serii, których prosta wypada w paśmie zmierzonym przez `LC`: detektor oparty o szynę (picDIV jeszcze niezsynchronizowany, faza poza oknem) daje serie wygięte o ok. +70 kodów i w buildzie 64 nauczyły go +20, po czym własne serie płytki były odrzucane (naprawione w buildzie 65). A od buildu 71 nadąża, gdy wygięcie się zmienia: na płytce autora rośnie ono, gdy odczyt spada, o ok. 0,19 kodu na kod, więc skok, który wyprowadza fazę o 60–100 ns na kilka minut, wygina każdą serię inaczej niż typowe, a typowe szło za nim najwyżej o jedną trzydziestą drugą kodu na sekundę — 131 serii odrzuconych jako wygięte w 16 minutach dwóch skoków z 27.10.

Teraz, gdy mediana wygięcia ostatnich 15 serii w paśmie odbiega od typowego o więcej niż 3,5 kodu, typowe od razu na nią przechodzi (na modelu tych minut 14–21 odrzuconych zamiast 270–290); wygięcie, które stoi w miejscu, jest śledzone jak dotąd. Zaraz po serii kondensator jest rozładowywany przez PA1 (`GPSDO_LTIC_ACTIVE_RESET`), więc następny impuls ładuje go od 0 V niezależnie od temperatury; sam upływ, wolniejszy na zimnej płytce, mógłby zostawić część ładunku z jednej sekundy pod odczytem następnej. Przed serią wypisuje liczniki od startu: czy zbocze zostało złapane, czy prosta się utrzymała (a jeśli nie, to dlaczego) i ile razy został poprzedni odczyt — a zostaje on, razem z qErr odbiornika, które do niego należy, do 3 s, gdy zbocza nie udało się złapać albo prosta została odrzucona, oraz gdy bramka odrzutów zatrzymuje skok; od buildu 71 także, w ilu seriach pominięto trzecią część i ile razy typowe wygięcie od razu się przesunęło (`every third conversion left out 12, typical bend moved 3`). `LRM 0` bierze zamiast tego medianę szesnastu wolniejszych pomiarów, jak buildy do 60 (do porównania; nie zapisuje się). Oba tryby odczytują upływ w innej chwili, więc kalibracja `LC` należy do trybu, w którym ją zrobiono: po przejściu na build 62 uruchom `LC` jeszcze raz.

Na czas serii — od 150 ms przed każdym zboczem do 2 ms po nim — linia poleceń wstrzymuje wypisywanie: tekst wychodzący przez Bluetooth w chwili zbocza zawyżał odczyt tej sekundy nawet o ~20 ns. Dłuższy wydruk, jak `LB`, wychodzi więc z krótką przerwą co sekundę; nic nie ginie.

3.4 Wybór algorytmu

- **`LA 12`** — akumulator wielopoziomowy (wg Alana Cashina). Sprawdzony wybór: trzyma fazę na kilku ns RMS, koryguje średnio co kilka minut. **To jest rekomendacja.**
- **`LA 11`** — ciągła pętla PI Larsa Waleniusa. Dojrzały, łagodny klasyk; znakomite długoterminowe zachowanie z jednym pokrętłem (`LTC`).
- **`LA 13`** — filtr Kalmana (część 4.6). Najnowszy: decyduje, ile uwierzyć każdemu odczytowi, z wariancji, a nie ze stałej, i bierze każdą potrzebną liczbę z `CT` i `LC`. Nie ma czego stroić. **Symulator go przemilczał** — na prawdziwym stole ogranicza go wolna wędrówka zera własnego detektora i na metryce odczytu fazy wypada za algorytmem 11 (patrz 4.6a, dlaczego ta metryka może być wobec niego niesprawiedliwa i jakie pytanie stoi otworem). Nowszy niż 12, mniej godzin na sprzęcie — rekomendacja wyżej zostaje; 13 jest tym, na który warto patrzeć.
- `LA 10` — trójstopniowa pętla fazowa (ACQ→DPLL→LOCK). Solidna, więcej parametrów.
- Algorytmy 0–9 to kolekcja historyczna — działają, są zamrożone, patrz Część 4.

`LA` bez argumentu pokazuje bieżący algorytm. Sam wybór **nie zapisuje się** — patrz krok 3.6.

3.5 Zalecane dodatki

- **`SAW 1`** — korekta piły. Błąd kwantyzacji impulsu 1PPS odbiornika (±8…±25 ns, raportowany co sekundę jako `qErr`) jest odejmowany od mierzonej fazy. Przy odbiorniku timingowym to darmowa dokładność; włącz.
- **`TZ <miasto>`** — czas lokalny z DST (np. `TZ Warsaw`, `TZ Adelaide`), albo `TO 1` dla stałego przesunięcia, `LT 1` żeby pokazywać czas lokalny zamiast UTC. `H TZ` tłumaczy format reguły, gdyby Twojej strefy zabrakło.
- **`PO <Pa>`** — offset dodawany do odczytu ciśnienia BMP280 (−5000..5000).
- **`AO <m>`** — offset dodawany do **wysokości z GPS** na wyświetlaczu i w telemetrii (−3000..3000 m). Koreguje pokazywaną wysokość do realnej rzeźby, np. gdy model geoidy myli się o 30 m: `AO 30`.

3.6 Zapis wszystkiego — `ES`

Wpisz `ES` i Enter. To zapisuje **wszystkie** ustawienia plus dane wyuczone do flash ringa w sektorze 7. Od tej chwili zanik zasilania niczego nie zabiera.

Garść komend-preferencji zapisuje **się sama** w chwili ustawienia i mówi Ci o tym — odpowiedź zawiera `[auto-saved: …]`. Są to: `TZ`, `TO`, `LT` (grupa strefy czasowej), `WU`, `SPL`, `SV` (flagi), `PO`, `AO` (offsety), a ponadto `CT` (sam zapisuje strojoną właśnie grupę PID) i zdany `LC` (zapisuje się do pierścienia live).

Wszystko, co dotyczy **pętli** — wzmocnienia (`KP/KI/KD/IL`), wszystkie parametry LTIC i Larsa, ustawienia algo-12, a nawet sam wybór `LA` — żyje w RAM dopóki nie zapiszesz. Każda taka komenda odpowiada podpowiedzią w stylu `[not saved – run 'ES LTIC' to keep it]`: wykonaj wskazany zapis grupowy, albo po prostu zapamiętaj: **po sesji strojenia — `ES`**.

3.7 Jak wygląda prawidłowa praca

- Słowo trendu (wyświetlacz + linia `PWM:` raportu) dochodzi do **`LOCK`** i tam zostaje — przy algo 12 krótkie błyski `CORR`/`ZC` są **normalne i zdrowe** (korekta to autotest algorytmu wg harmonogramu; wyświetlacz i tak liczy je jako zablokowane).
- Faza (`dph:` w raporcie) błądzi wokół zera w dziesiątkach ns i zawsze wraca — nie może rampować w dal.
- Okna częstotliwości się domykają: `100s:` w milihercach, `1ks:` w mikrohercach, w godzinę–dwie.
- `CS` (statystyka korekt) wypisuje małe, stabilne wartości RMS, które nie rosną z godziną na godzinę.
- Na TFT duża liczba częstotliwości jest **zielona**.

Na spokojnym stole z algo 12 oczekuj: faza 5–20 ns RMS, korekty po kilka LSB co kilka minut, dewiacja Allana w klasie 1e-11…1e-12 od 1000 s w górę. Przy grzejniku, otwieranych drzwiach albo pełnym słońcu liczby będą gorsze — to fizyka, nie błąd.

Część 4 — Czternaście algorytmów (0–13) i ich parametry

Jedna idea leży u podstaw wszystkiego: firmware liczy częstotliwość OCXO licznikiem bramkowanym (TIM2), mierzy fazę GPS-vs-OCXO detektorem LTIC i koryguje napięcie sterujące („DAC" PWM, 65536 kroków, ~48,8 µV na krok przy 16 bitach; z `GPSDO_PWM_DITHER` efektywna rozdzielczość to 24 bity — około 0,2 µV na krok). Konwencja znaku: mierzony błąd `e = freq − 10 MHz`; gdy `e > 0`, oscylator jest szybki i PWM musi iść **w dół** (dla EFC o dodatniej czułości; `LPOL` / obsługa polaryzacji obejmuje odwrócone EFC).

> Jeśli faza, błąd częstotliwości albo PID są dla Ciebie nowymi słowami — zatrzymaj się i przeczytaj Aneks A (za czym pędzi pętla i dlaczego musi być delikatna) oraz Aneks B (co naprawdę robią litery P, I i D). Dziesięć minut tam czyni każdy parametr poniżej czytelnym.

Linia `Learn:` w raporcie nazywa aktywny algorytm; tuner automatycznie dobiera z niego rodzinę wykresów.

4.0 Menu

#	Nazwa (jak pokazywana)	Jedno zdanie	Status
0	primitive	pierwotny sterownik krokowy André Balsy, cykl 429 s	domyślny po zimnym flashu
1	forced-drift	+1 LSB na 1000 s — charakteryzacja oscylatora	diagnostyczny
2	random-walk	szum ±1 LSB co 5 s — pomiar podłogi szumu	diagnostyczny
3	FLL-PID-man	PID na średniej 100 s	klasyk
4	PLL-PI-man	PI na fazie, cykl 10 s	klasyk
5	PLL-PID-man	jak 4, z własnym slotem strojenia	klasyk
6	FLL-PID-gen	FLL PID, strojenie genetyczne	klasyk
7	PLL-PID-gen	stary roboczy PLL; jego Kp przechowuje wynik CT	klasyk
8	hybrid-FLL-PLL	sigmoidalne mieszanie 6 i 7 wg wielkości błędu	klasyk
9	NN-MLP	mała sieć neuronowa + wyuczony dryf termiczny w holdover	klasyk
10	LTIC-3stage	maszyna stanów ACQ→DPLL→LOCK na detektorze fazy	linia rekomendowana
11	LTIC-Lars	ciągła pętla PI Larsa Waleniusa	linia rekomendowana
12	multi-level	akumulator wielopoziomowy Alana Cashina	**rekomendacja**
13	kalman	trójstanowy filtr Kalmana — faza, częstotliwość, starzenie	najnowszy

Algorytmy 0–9 są **zamrożone**: zostały, bo działają i bo slot strojenia algorytmu 7 pełni dodatkowo rolę magazynu wyniku kalibracji CT. Nowy rozwój dotyczy wyłącznie 10/11/12/13.

Wybór: `LA <n>` (trwały przez `ES ALGO`). Świeża jednostka zawsze startuje z algorytmem 0 — wybierz swój raz, zapisz, i już zostaje na zawsze.

4.1 Algorytmy 0–2 (diagnostyczne)

Bez parametrów. 0 koryguje PWM, gdy średnie częstotliwości przekraczają stałe progi. 1 prowadzi PWM liniowo (charakteryzacja EFC). 2 wstrzykuje szum (pomiar podłogi pętli). Do normalnej eksploatacji nie są potrzebne.

4.2 Algorytmy 3–9 (klasyki) — parametry przez KP/KI/KD/IL

`KP <algo> <val>` / `KI` / `KD` ustawia wzmocnienia (algorytmy 3–7, 0..100000); `IL <algo> <val>` ogranicznik integratora (algorytmy 3–9, 100..100000). `LP [n]` wypisuje. Wartości domyślne (a także baza, od której `CT` liczy własne, oparte na pomiarze):

algo	Kp	Ki	Kd	I_LIMIT
3	70,0	0,70	175,0	9000
4	1000	0,020	2,0	7000
5	1000	0,020	2,0	10000
6	205	0,264	14950	13000
7	1000	0,020	2,0	10000
8	—	—	—	13000
9	—	—	—	450

Dodatki: algorytm 8 ma `BC` (punkt przecięcia mieszania, domyślnie 0,024 Hz) i `BS` (skala mieszania, 0,012 Hz) — środek i szerokość sigmoidy decydujące, ile FLC a ile PLL wmieszać przy danym błędzie; algorytm 9 ma `NS` (maksymalny krok, domyślnie 175 LSB). Zapis: `ES PID`. Szczerze: po `CT` nie powinieneś musieć tego ruszać — po to jest CT.

4.3 Algorytm 10 — LTIC trójstopniowy

Maszyna stanów na detektorze fazy: **ACQ** (ściąganie po częstotliwości, do tego centrowanie rampy detektora) → **DPLL** (cykl 2 s, częstotliwość + faza) → **LOCK** (korekty co `LIV` sekund z oszacowania pary okien).

Parametry (wszystkie `ES LTIC`; `LL` wypisuje całość):

Komenda	Znaczenie	Zakres / domyślne
`LAT`	próg fazy dla ACQ	ns, domyślnie 100
`LDT`	próg dryfu DPLL→LOCK	1e-13..1,0, domyślnie 5e-10
`LIV`	odstęp korekt w LOCK	1..600 s, domyślnie 300
`AQP/AQI/AQD/AQL`	PID stopnia ACQ	0..100000
`DPP/DPI/DPD/DPL`	PID stopnia DPLL	0..100000
`LKP/LKI/LKD/LKL`	PID stopnia LOCK	0..100000
`LPOL`	polaryzacja PWM→faza, −1/0/+1	0 = nieustawiona: pętla czeka
`LCV`	cel centrowania ACQ	0 (= auto, środek zakresu) .. 3,3
`ACG g [cap]`	wzmocnienie [limit kroku] napędu centrowania	50..20000 LSB/V [5..1000 LSB]
`FA/FAD/FAL`	okno uśredniania częstotliwości dla członu tłumiącego (oba / DPLL / LOCK)	10, 100 albo 1000 s

`ltic_autotune` (część CT/LC) wypełnia wzmocnienia stopni na podstawie zmierzonej czułości, więc opisana sekwencja startowa już ten algorytm stroi. Jeśli ACQ nigdy nie wychodzi, sprawdź `LPOL` — pętla ostrzega i czeka, aż ustawisz.

4.4 Algorytm 11 — LTIC-Lars, ciągła pętla PI

Pętla Larsa Waleniusa, portowana wiernie. Jeden regulator PI liczony co sekundę, z adaptacyjnym pre-filtrem; lock, gdy przefiltrowana faza mieści się w `LPL` ns przez `LTC × LPF` sekund. Detektor na krańcu → automatyczne ściągnięcie po częstotliwości, jedno dozwolone ponowne zbrojenie picDIV.

Komenda	Znaczenie	Zakres / domyślne
`LG`	wzmocnienie pętli. **0 = auto z CT** (zalecane)	0..10000
`LD`	tłumienie	>0..1000, domyślnie 3
`LTC`	stała czasowa pętli — **jedyne istotne pokrętło**	1..600 s, domyślnie 60
`LFD`	dzielnik pre-filtra (pre-filter = LTC/to)	1..100, domyślnie 2
`LTO`	cel fazy na detektorze	0..3,300 V, domyślnie 2,111 V

Komenda	Znaczenie	Zakres / domyślne
`LPL`	okno fazy dla locka	1..10000 ns, domyślnie 100
`LPF`	współczynnik trzymania locka (hold = LPF × LTC)	1..100, domyślnie 5
`LTK`	sprzężenie termiczne, kroki DAC na krok ADC	−32000..32000, 0 = wyłączone
`LTR`	odniesienie temperatury	0..3,300 V

Strojenie w jednym zdaniu: zostaw `LG 0`, ustaw `LTC` na to, jak delikatnie pętla ma prowadzić (60 s to dobry start; 240 s uspokaja zakłócone miejsce; krócej to tylko więcej szumu), a resztę zostaw domyślnie. Wszystko przez `ES LTIC`. Zakładka **LTIC-Lars** w tunerze wystawia dokładnie te pola.

4.5 Algorytm 12 — akumulator wielopoziomowy

Port z GPSDO Alana Cashina (MIS42N), dopracowany w v1.04–v1.06. Zero stałego cyklu: **o czasie uśredniania decyduje wielkość błędu.**

Jak to działa, prostymi słowami. Co sekundę odczyt fazy wchodzi do drabiny akumulatorów. Poziom *n* uśrednia po 2^(n+1) sekund — drabina biegnie 2, 4, 8, … 2048 s (11 poziomów). Na każdym poziomie siedzi para wartości (A = starsza połowa, B = nowsza). Gdy para się domyka, liczone są dwie liczby: **nachylenie** (B − A = błąd częstotliwości na tym przedziale) i **faza ekstrapolowana** (3B − A = faza właśnie teraz). Jeśli faza mieści się w limicie poziomu, para jest sumowana i awansuje o poziom wyżej (podwójne uśrednianie). Jeśli przekracza limit — korekta strzela natychmiast, drabina się resetuje, a korekta rozkłada się delikatnie na span, z którego przyszła (minimum 64 s — „GPSDO chce duży błąd korygować łagodnie"; u Alana podłoga wynosi 16 s, dobrana tak, by korekta w rozgrzewce nie zażądała napięcia sterującego poza zakresem DAC — mniejsza podłoga to większe kroki, większa to wolniejszy powrót fazy). Niezależnie od tego, osiągnięcie poziomu `MR` (domyślnie 7 = 256 s) wymusza korektę nawet pod wszystkimi limitami — inaczej powolny dryf nigdy nie doczekałby się reakcji.

Każda korekta ma do trzech części i to jest celowe: skorygować zmierzony błąd **częstotliwości**, sprowadzić **fazę** do zera, i nigdy jedno bez drugiego. Potem trik, który Alan uważa za istotę: wykonany celowo slewing jest zapamiętany, a przy pierwszym odczycie fazy przekraczającym zero w oczekiwanym kierunku dokładnie ten slewing jest zdejmowany (**ZC**, zniesienie w przejściu przez zero) — oscylator zostaje z właściwą częstotliwością *i* bez błędu fazy.

Test przejścia przez zero jest uzbrajany **tylko po korekcie wywołanej przez limit** — to reguła Alana. Korekta planowa (poziom `MR`) pada z zegara, przy fazie tam, gdzie akurat jest, więc nie ma przeregulowania, na które trzeba by czekać; a samo ZC się nie uzbraja ponownie, bo ZC *jest* zniesieniem. Uzbrojenie w którymkolwiek z tych przypadków pozwoliłoby niezwiązanemu przejściu przez zero kilka minut później wyciągnąć krok z nieaktualnego nachylenia.

Trendy właściwe dla 12: `WAIT` (brak danych) → `SYNC` (5 s stabilizacji po zbrojeniu dzielnika) → `FLL` (detektor na krańcu, ściąganie częstotliwości) → `NOPH` (brak ważnej fazy, PWM zamrożony) → `ACQ` → `LOCK` (>16 s prawdziwego spokoju **i** częstotliwość w bramce; korekty i ZC *nie* zerują licznika spokoju — one są dowodem zdrowia, nie szumu). Przejściowe błyski `CORR` / `ZC` są normalne. `NoCT` znaczy: wzmocnienie jest auto, ale `CT` nigdy nie biegło — uruchom `CT`. `NoPL` znaczy: polaryzacja EFC nieustawiona — pętla czeka, aż ustawisz `LPOL -1` albo `1`.

Parametry (`ES ALGO12`; `ML` wypisuje wszystko, łącznie z bieżącym szacunkiem szumu 1-sigma):

Komenda	Znaczenie	Zakres / domyślne
`MG`	wzmocnienie, LSB na ns fazy. **0 = auto z CT**	0..10000, domyślnie 0 (auto)
`MR`	poziom wymuszający korektę	0..10, domyślnie 7 (256 s)
`MM`	jak korekta dochodzi do pinu: 1 = spokojny (niżej), 0 = pierwotna ścieżka	0..1, domyślnie 1
`MF`	skąd limity per poziom: 0=za MG, 1=tabela zapisana, 2=wzór sigmowy, 3=zmierzony fit	0..3, domyślnie 0
`MFT`	przy MF 3: docelowe sekundy między korektami szumowymi	0 (=3600) albo 2048..65535 s

Komenda	Znaczenie	Zakres / domyślne
`MLP n v`	jeden wiersz 11-poziomowej tabeli limitów	poziom 0..10, wartość w nanosekundach

Tryb spokojny (`MM 1`, domyślny od buildu 60). Gdy faza jest już w `LAT`, zmieniają się dwie rzeczy, a arytmetyka hierarchii nie: od poziomu 128 s w górę człon częstotliwości idzie w całości zamiast połowy; a wyjście dochodzi do korekty najwyżej o 1 LSB na sekundę (zmiana ponad 32 LSB — akwizycja, FLL, uzbrojenie — nadal od razu), przy czym przejście przez zero zdejmuje dokładnie ten slew, który założyło, razem z ułamkiem. Build 60 kazał też poziomom poniżej tej granicy czekać na trzykrotność limitu i wiązał granicę z `MR`. Z tym płytka dała 6,1 ns przy `MR 7` i 9,5 przy `MR 10`, wobec 2,7 dla algorytmu 11 w tym samym tygodniu; w symulacji skalibrowanej na tych przebiegach bez nich wychodzi 5,2 ns przy `MR 7` i 5,1 przy `MR 10` (`MM 0`: 6,1 i 5,6), więc build 61 usunął jedno i drugie. PWM rusza się pięć razy mniej niż przy algorytmie 11, który nadal trzyma fazę mniej więcej dwa razy ciaśniej; impulsy to sposób, w jaki ten algorytm przesuwa fazę. Poniżej `MR 6` żadna korekta nie sięga 128 s i człon częstotliwości zostaje połowiony. `MM 0` to pierwotna ścieżka, bez zmian, do porównania A/B na jednej płytce.

Domyślna tabela limitów to tabela Alana (przeskalowana z jego detektora 25 ns na ten ~1 ns). W nanosekundach na span: L0 462, L1 400, L2 331, L3 264, L4 191, L5 164, L6 126, L7 117, L8 108, L9 103, L10 63 ns. `ML` oznacza ją **UNTUNED** (tylko wiersz 128 s pochodził kiedyś ze specyfikacji) — traktuj jako sprawdzony punkt startowy, nie świętość. Jej geneza, słowami samego Alana, to **najgorszy scenariusz odbioru**: liczby policzono dla NEO-6 z anteną w pomieszczeniu, którego 1PPS potrafi mylić o ±100 ns przez krótkie okresy (średnio ±30–40 ns). Tabela jest celowo luźna — wymierzona w najgorszy odbiornik, z jakim ten projekt może mieć do czynienia, a nie w sprzęt klasy LEA-M8T, który ta konstrukcja zwykle ma.

Które MF wybrać? Wyniki z pola, dwie płytki:

- **Spokojne miejsce (antena na parapecie w domu, stabilna temperatura):** wzór sigmowy (`MF 2`) albo pomiar (`MF 3`) działa — pętla sama skaluje się i koryguje co kilka minut.
- **Zakłócone miejsce (warsztat z drzwiami, blisko grzejnika, ruch powietrza):** autoskalowanie może polować na poziomie 0 (limity ciaśniejsze niż realne wędrówki środowiska). Tam luźna tabela Alana (`MF 1`) była dramatycznie stabilniejsza. Jeśli widzisz ciągłe korekty poziomu 0 — przełącz na `MF 1`.

Za oboma wynikami stoi zadeklarowana zasada projektowa Alana, warta poznania przed strojeniem: *„nie chcemy, żeby algorytm działał limitami, chcemy, żeby działał korektami wg harmonogramu w określonych odstępach*". Korekta z poziomu run (`MR`) ma być koniem pociągowym, a limity tylko siatką bezpieczeństwa na rozgrzewkę i zaniki odbioru — i właśnie dlatego jego tabela jest luźna. Ciasna tabela strzelająca bez przerwy działa według tej filozofii przeciw algorytmowi, nie z nim.

Wzmocnienie (`MG`) należy do **oscylatora**; limity należą do **szumu miejsca** — właśnie dlatego ustawiają je różne komendy i właśnie po to jest `MF`, żeby dało się wyrazić „zmierzone wzmocnienie, ręczne limity".

W tunerze: zakładka **Multi-level (algo 12)** ma cztery skalary i całą 11-wierszową tabelę limitów z przyciskami Send/Read/List, a wykresy na żywo same przechodzą na rodzinę algo-12 (`ph` błąd fazy, poziom, licznik korekt, sigma, licznik ZC).

4.6 Algorytm 13 — filtr Kalmana (autorski)

Algorytmy 10, 11 i 12 odpowiadają na to samo pytanie — *ile z dzisiejszego odczytu fazy mam uwierzyć?* — liczbą ustaloną z góry: etapem maszyny stanów, stałą czasową, poziomem w tabeli progów. Ten odpowiada na nie z wariancji, i odpowiada od nowa co sekundę.

Niesie trzy stany — **fazę**, **częstotliwość**, **starzenie** — każdy z kowariancją. Wie, jak głośny jest Twój detektor, bo to mierzy, i jak szybko błądzi Twój oscylator, bo to też mierzy — więc waga każdego odczytu jest tym, co te dwie liczby aktualnie mówią. Przy krótkich czasach uśredniania, gdzie detektor jest głośny a OCXO spokojny, opiera się na oscylatorze; przy długich, gdzie OCXO odchodzi, ustępuje GPS-owi.

Ile kosztuje. Trzy stany i pomiar **skalarny**, więc podręcznikowe odwracanie macierzy to jedno dzielenie: około 140 mnożeń z akumulacją i 36 bajtów, raz na sekundę — jakaś mikrosekunda M4F. Twierdzenie, że Kalman wymaga Cortexa-

A albo FPGA, dotyczy dwudziestostanowych filtrów GNSS, nie tego.

Holdover wypada za darmo. Stan niesie częstotliwość i starzenie razem z ich kowariancjami, więc utrata fazy nie jest przypadkiem szczególnym: filtr przestaje aktualizować, dalej przewiduje i dalej steruje. Trend pokazuje `NOPH`.

Korzysta z dwóch pomiarów. Fazy z detektora LTIC i częstotliwości z bramkowanego licznika TIM2. To drugie nie jest ozdobnikiem: przy niezsynchronizowanym picDIV nie ma poprawnej fazy w ogóle, a filtr mający tylko pomiar fazy nie ma wtedy **żadnego** pomiaru — przewiduje ze stanu wciąż wyzerowanego, a oscylator odchodzi. TIM2 jest też tym, co pozwala sprawdzić sam detektor: faza i częstotliwość to ta sama wielkość zróżniczkowana, więc w oknie faza **musi** przejechać tyle, ile wynosi suma błędu częstotliwości. Detektor, który się nie rusza, choć TIM2 mówi, że musi, nic nie mierzy — pętla uzbraja picDIV i steruje na samej częstotliwości, póki się nie poprawi.

Wszystko wyprowadzone z `CT` i `LC`. Nic w nim nie jest stałą zmierzoną na cudzym stole: własny kwant detektora (`ns_per_volt × 3,3/4096`) daje zasiew szumu pomiarowego i jego podłogę, pół pasma detektora daje kowariancję na zimnym starcie, pasmo przejechane w jednym horyzoncie daje tę dla częstotliwości, a `R/KT³` zasiewa szum procesu. Po ponownym `CT` albo `LC` liczby idą za tym w ciągu sekundy. Przy starcie filtr wypisuje jedną linię z tym, co ustalił:

```
KAL: from CT/LC  res 1.01ns  R0 2.52ns  Q0 0.000006600  P0 1500ns  lim 939LSB  arm<0.25Hz
```

Dwa szumy procesowe, nie jeden. Model zegara, który ten filtr realizuje, niesie `Sf` — biały szum częstotliwości, objawiający się jako błądzenie przypadkowe fazy — oraz `Sg`, błądzenie przypadkowe częstotliwości: `Q = [[Sf·t + Sg·t³/3, Sg·t²/2], [Sg·t²/2, Sg·t]]`. Do 01.09 istniało tu tylko `Sg`, i to jedno pominięcie było całym przesterowaniem: bez `Sf` jedynym sposobem wyjaśnienia fazy, która ruszyła się bardziej, niż przewidziano, jest orzeczenie, że błądzi *częstotliwość* — co podnosi wzmocnienie zapisujące stan częstotliwości, za którym jedzie DAC. `Sf` jest teraz mierzone, a nie adaptowane: różnice fazy na opóźnieniu 1 i 16 niosą `sigma_R² + Sf·k/2`, więc dwa opóźnienia rozdzielają biały szum od błądzenia tam, gdzie jedno nie potrafi — a różnica, która nie przekracza dwóch sigm własnego szumu estymatora, jest raportowana jako zero, co przy czystym detektorze jest właściwą odpowiedzią. `KL` pokazuje ją obok `R` i `Q`.

Q się adaptuje, ale nie może wyprzedzić swojego horyzontu. Filtr dostraja szum procesu `Q` tak, aby jego własne innowacje wychodziły tej wielkości, jakiej się spodziewał. Ma to jedno systematyczne obciążenie i na stole ono uderzyło: gdy błąd detektora jest *skorelowany* — zero wędrujące w skali minut — innowacje są większe, niż filtr przewiduje, z powodu niemającego nic wspólnego z oscylatorem, więc `Q` narasta sekunda po sekundzie, aż kowariancja urośnie na tyle, by je wytłumaczyć. Zatrzymuje się, ale zatrzymuje się wysoko. 30.08 płytka pracowała z `Q` 195x powyżej zasiewu: filtr pięć razy szybszy niż horyzont, nad którym kazano mu sterować, ruszający DAC-iem o 1,80 LSB na sekundę wobec 0,49 algorytmu 11 tej samej nocy — i bez lepszej fazy w zamian.

`(R/Q)^(1/3)` ma wymiar czasu i jest własną stałą czasową filtru; `Q = R/KT³` to dokładnie stwierdzenie *biegnij tak szybko jak zadany horyzont*. Tam właśnie adaptacja jest teraz ograniczona: **filtr nie może biec szybciej niż `KT`**, przy `R` branym z pomiaru, a nie z zasiewu — i to sprawia, że reguła znaczy to samo na każdej płytce. Zmierzone na dwóch obiektach, pięciu ziarnach szumu i trzech poziomach wędrówki detektora: ADEV dla krótkich tau dwa razy lepsze, ruch DAC-a 2,4× mniejszy, kosztem 7% na sd fazy. Nic nie tracimy, odmawiając `Q` pochłaniania skorelowanego błędu detektora, bo `R` już go ma: `R` jest mierzone z różnic branych z szesnastosekundowym opóźnieniem, więc zero wędrujące w tych skalach siedzi w szumie pomiarowym, gdzie jego miejsce. Chcesz szybszej pętli? Skróć `KT` — teraz to jedyna rzecz, która nią rusza. Wartość przypięta ręcznie przez `KQ` przechodzi bez zmian: ograniczenie dotyczy adaptacji, nie Ciebie. `KL` pisze `[at ceiling]`, gdy `Q` opiera się o sufit. Samo to nie znaczy, że `KT` jest za długie, choć do buildu 70 tak mówiły ta instrukcja `KL`: to, co pcha Q w górę, pochodzi równie często z powolnego błądzenia po stronie GPS albo detektora, co z oscylatora — a szybsza pętla je kopiuje — i na płytce Dana Wieringa (28–29.10) `KT 1000` siedziało pod sufitem i mimo to wypadło lepiej niż `KT 500` — ADEV płaskie na $2{,}9\cdot10^{-12}$ od 40 do 400 s, wobec garbu $4\cdot10^{-12}$ przy 100–150 s. Wartości `KT` porównuj po ADEV, po przebiegu na każdą.

Parametry — wszystkie cztery są opcjonalne, a domyślne są tymi, których się używa:

Komenda	Domyślnie	Znaczenie
`KR [ns]`	`0` = mierz	Szum pomiaru. Zero znaczy „zmierz go z różnic samego detektora o szesnastosekundowym rozstawie" — biały szum i wolna wędrówka zera razem, i tego właśnie chcesz. Ustawiaj tylko, żeby unieruchomić filtr na czas eksperymentu.

Komenda	Domyślnie	Znaczenie
`KQ [v]`	`0` = adaptuj	Szum procesu, $(ns/s)^2$ na sekundę. Zero znaczy „adaptuj z ciągu innowacji".
`KT [s]`	`100`	Horyzont fazy: jak szybko sterowanie zeruje estymatę fazy. Krócej — mocniej za GPS, dłużej — bardziej na oscylatorze. Ustawia też cierpliwość testu zawieszenia: pięć horyzontów daleko od zera i picDIV zostaje uzbrojony. Od buildu 67 zimny start używa z niego najwyżej 100 s (zob. `KT` niżej).
`KL`	—	Wypisuje stan: fazę, częstotliwość i starzenie, jak mocno w każde wierzy, aktualne R i Q oraz ile odczytów odrzuciła bramka innowacji. `[at ceiling]` przy Q oznacza, że adaptacja opiera się o swoje ograniczenie.

`KR`, `KQ` i `KT` zapisują się w chwili wpisania, we własnym rekordzie flash ringu — bez `ES`, a starszy firmware po prostu nigdy o ten rekord nie pyta.

Linia Learn wygląda tak:

```
Learn: algo=13 (kalman) ph=-12.4ns f=-118.30ps/s sig=2.48ns R=2.64 rej=3 arm=1
```

— faza i częstotliwość, w które filtr **wierzy** (a nie odczyt z tej sekundy, bo o to właśnie chodzi w posiadaniu filtru), jak mocno wierzy w fazę, zmierzony szum pomiaru, ile odczytów odrzuciła bramka innowacji i ile razy uzbroił dzielnik. `HOLD=<s>` pojawia się, gdy pętla jedzie na samym modelu, a `KF`, gdy zakłócenie jest domykane horyzontem `KF` (build 69).

W tunerze: górny wykres pokazuje estymatę fazy z filtru, a pod nią `Vphase` z prowadnicami pasma — bo pytanie, które ta pętla stawia najczęściej, brzmi: czy detektor w ogóle żyje.

Zmierzono na symulatorze (`tools/loopsim`, pięć ziaren szumu, dwie plansze, biały szum detektora): sd fazy **0,81 / 1,20 ns**, wobec 3,07 / 4,20 dla algorytmu 12 i 3,62 / 7,39 dla algorytmu 11. **Stół się nie zgadza i stół ma rację**: replay tych samych planów z wolną wędrówką zera odwraca kolejność (11: 7,45 → 10,07 ns; 13: 1,22 → 7,23), a na prawdziwym sprzęcie 29–30.08 ta sama płytka dała 2,96 ns (11) wobec 5,2–5,9 ns (13) z płaską podłogą na średnich czasach uśredniania — poziom wędrówki własnego detektora, niezależny od pasma pętli. Szum symulatora był zbyt biały; traktuj jego liczby jako względne, nigdy bezwzględne (4.6a).

4.6a Jak przebiega sekunda pętli — i co poruszają pokrętła

Jedna sekunda, po kolei. *Predykcja*: stan (faza, częstotliwość, starzenie) przepychany o sekundę. Potem najwyżej dwa *pomiary* korygują — faza z detektora LTIC (chyba że na szynie, zamrożona lub poza pasmem) i częstotliwość z TIM2 (zawsze; utrzymuje pętlę przy życiu, gdy detektor jest ślepy). Na koniec *sterowanie*: `u = -(freq + faza/KC)` — skasuj estymowany błąd częstotliwości i domknij fazę po horyzoncie **sterownika** `KC` (od pierwszego zablokowania pętli; wcześniej, w akwizycji, wolnym krokiem `KT`), z ograniczeniem do pasma detektora; reszta sub-LSB idzie na następną sekundę. Korekta może rosnąć tylko stopniowo (jej największy dozwolony rozmiar rozłożony na jeden horyzont), żeby restart nie szarpnął oscylatorem, ale maleć może tak szybko, jak chce filtr — przed buildem 65 nie mogła, i dociąganie z odległości pół herca rozhuśtało PWM od szyny do szyny (patrz lista zmian). To, co doszło do pinu, wraca do stanu częstotliwości. Wokół rdzenia: bramka 4σ, test zaufania (czy faza rusza tak, jak każe TIM2? — z ignorowaniem mrugnięć mniejszych niż własna rozdzielczość licznika), cztery sekundy celowej ciszy fazy po uzbrojeniu dzielnika (odczyty z rampy wtedy nie są fazą), startowe uzbrojenie referencji i holdover.

Insight uczciwego R. R, które pokazuje `KL` (~2,9 ns na tym stanowisku), to pomiar z własnych różnic detektora o szesnastosekundowym rozstawie: podłoga biała (~2,5 ns) plus wzrost wolnej wędrówki zera przez te 16 s. **Pełna** wędrówka zera to osobna struktura (~2,6 ns w skali ~45 s na tym stanowisku), której filtr **celowo nie ściga** — dlatego algorytm 13 odmawia gonienia za wolną strukturą detektora, za którą algorytm 11 idzie bez pytania. Zmierzone: przy spokojnej pętli dph pokazuje płaską podłogę 5–9 ns od 10 s do 600 s uśredniania **niezależnie od pasma pętli** — szerokie, adaptowane i sztywne lądują na tym samym poziomie, czyli poziomie wędrówki detektora. Czy gonienie tej wędrówki (11), czy odmawianie (13) daje lepsze *wyjście*, rozstrzygnie tylko wzorzec niezależny.

`KR [ns]` (domyślnie 0 = mierz). Przypięcie do podłogi białej (`KR 2.5`) każe filtrowi podążać za detektorem w pełni: sd@600 poprawiło się ~30% w spokojnych godzinach, ale bramka 4σ zawęża się z R, więc zdarzenia GPS (dziesiątki ns)

są odrzucane — odrzucenia wzrosły ze 163 do 5021 na noc. Przypinaj tylko do obłożenia eksperymentu; `KR 0` jest lepszym domyślnym.

`KQ [v]` (domyślnie 0 = adaptuj). Adaptacja działała jak zapadka: skorelowany błąd detektora sprawia, że innowacje są większe, niż przewiduje kowariancja, więc **Q rosło** — 195× seed po jednej nocy, czyli filtr pięć razy szybszy niż jego horyzont, ruszający DAC-iem o 1,80 LSB/s wobec 0,49 algorytmu 11. Teraz jest ograniczone do R/KT^3 — **filtr nie może biec szybciej niż `KT`** — co zmniejsza ruch DAC-a 2,4× i podwaja ADEV dla krótkich tau (4.6). Płytka Dana Wieringa, ten sam firmware, nic nie ruszane, doszła do samej starej szyny: `Q` 1000× zasiewu w 2h37m, DAC ruszający się 11,05 LSB/s i ADEV na 20 s równe 8,6e-11 wobec 3,1e-12 dla algo 11 na tej samej płytce i tym samym rubidzie. `KL` pisze `[at ceiling]`, gdy adaptacja opiera się o ograniczenie. Przypięcie na zasiewie (`KQ 0.000006359` — dokładna wartość z linii `KAL: from CT/LC`) usztywnia pętlę jeszcze bardziej i niemal zamraża PWM; wartość przypięta ręcznie przechodzi bez zmian, ograniczenie dotyczy adaptacji. **KQ zapisuje się od wpisania** — wróć przez `KQ 0`.

`KT [s]` (domyślnie 100; w buildzie 60 było 200). Od buildu 36 to horyzont **estymatora** — cierpliwość rozumu, nie tempo rąk. Krótszy KT = filtr o większym pasmie wiary (i wyższy sufit adaptacji Q, bo R/KT^3); dłuższy = oparcie na stabilności oscylatora. To też cierpliwość czujnika zastoju — pięć horyzontów daleko i picDIV jest uzbrajany. Tempem domykania fazy steruje już `KC`. Która strona 100 s jest właściwa, zależy od oscylatora: OCXO autora błądzi szybko i chce 100 s; cichy OCXO, mierzony względem rubidu, może chcieć 300–1000 s, a komunikat komendy `KT` powyżej 200 s mówi, jak wybierać: po ADEV, po przebiegu na każdą wartość — nie po Q pod sufitem w `KL` (build 71; do buildu 70 komunikat kazał wtedy skrócić `KT`, a na płytce Dana `KT 1000` siedziało pod sufitem i pobiło `KT 500`). Od buildu 67 dociąganie używa `KT` tylko do 100 s — horyzont sterowania, pierwsze Q, założenie o częstotliwości, bramka uzbrojenia — więc długi `KT` nie kosztuje już zimnego startu: na modelu cichej płytki `KT 1000` lockował wcześniej w 28–41 minut, teraz w 6–12, a `KT 100` dokładnie jak przedtem.

`KC [s]` (domyślnie 0 = auto = KT/3, 33 s przy domyślnym `KT` — w buildach 61–67 KT/6, w buildzie 60 KT, wcześniej KT/3; **skuteczne dopiero po pierwszym zablokowaniu pętli** — w akwizycji i po restarcie algorytmu sterowanie wraca do KT, od buildu 67 najwyżej 100 s). Jak szybko *ręce* domykają fazę, którą *rozum* już zna: pomysł polega na tym, że estymat fazy jest już wygładzony, więc szybkie jego domykanie nie wzmacnia szumu — wzmacnia tylko poprawkę. Zmierzone na 20-godzinnym logu: ADEV wyjścia lepsze o ~23% na tau 256–4096 przy ditherze DAC rosnącym z 0,37 do 0,52 LSB/s — to budżet, nie darmo. **Oceniaj to po oscylatorze, nie po logowanej fazie.** Logowana faza jest odczytem detektora, a szybkie ręce poprawiają ją, przepisując do oscylatora własne powolne błądzenie detektora — a z płytki wychodzi oscylator. To, co robi oscylator, to scałkowane PWM: jego kroki razy herce na LSB; ADEV tego iloczynu zgadzało się co do kilku procent od 4 do 400 s z pomiarem płytki Dana względem wzorca rubidowego (14.10). Na płytce autora, w jednej sesji, noc przy `KC 16`, a potem dwie noce i dzień przy `KC 33`: PWM zmieniało się 664–744 razy na godzinę zamiast 1349, a to ADEV było 1,7 razy niższe przy 4–16 s, o jedną piątą niższe przy 64 s i takie samo przy 256 s, za jedną piątą więcej logowanej fazy (odch. std. 2,14–2,30 ns wobec 1,83) — to domyślne od buildu 68. KT/6 (buildy 61–67) wybrano wyłącznie po logowanej fazie: noc przy `KC 16` wobec algorytmu 11 noc wcześniej, 2,63 wobec 2,73 ns, przy PWM zmieniającym się ok. 2,5 razy częściej, krokami 1–4 LSB. `KT 200` / `KC` = `KT` z buildu 60 dało 5,62 ns: dobrano je na symulacji bez szybkiego szumu samego oscylatora, a tak wolny regulator ten szum przepuszcza. W symulacji skalibrowanej na płytce `KC 25` daje mniej więcej fazę algorytmu 11 przy 1,4 raza większym ruchu DAC, a poniżej ~15 s faza zyskuje niewiele, za to DAC rusza się dużo więcej. (Wcześniejszy sweep, który uznał długie `KC` za niestabilne, szedł na symulatorze źle czytającym ułamek DAC — patrz changelog buildu 60.) Płytka zachowuje zapisane, a wpisanie któregokolwiek z `KR`, `KQ`, `KT`, `KC`, `KF` zapisuje wszystkie pięć — płytka, na której w buildzie 60 wpisano którekolwiek, wstanie z `KT 200`; `KT 100` + `KC 0` przywracają domyślne. Sam `KC` bez argumentu pokazuje wartość ustawioną i tę faktycznie w mocy.

`KF [s]` (build 69; domyślnie 16 s, 10–250 s, `KF 0` = wyłączone). Horyzont, którego sterownik używa, dopóki trwa **zakłócenie**. `KC` to wybór co do szumu; skok częstotliwości oscylatora — spadek zasilania, uderzenie, skok kwarcu — szumem nie jest, a wolny sterownik pozwala mu odepchnąć fazę odpowiednio dalej. 21.10 trzyminutowy spadek szyny 5 V na płytce autora (do 4,37 V) przestawił OCXO o $2{,}7 \cdot 10^{-9}$: przy `KC 33` faza poszła do +75 ns, po powrocie napięcia do −51 ns i wróciła w ±10 ns dopiero po 12 minutach; to samo odtworzone z `KF 16` daje +41/−29 ns i 5½ minuty. Tryb włącza się, gdy estymata fazy filtru przekroczy 10-krotność spokojnego szumu płytki (20 ns u autora, około 37 ns na detektorze tak szumiącym jak Dana) albo gdy bramka puści po dziesięciu odrzuconych odczytach, a wyłącza po dwóch minutach z rzędu poniżej 3-krotności tego szumu. Działa na estymacie, nie na odczycie, nigdy nie robi sterownika wolniejszym niż `KC`, a po starcie albo własnym uzbrojeniu picDIV czeka, aż faza się uspokoi (`AP` wpisane w konsoli liczy

się jako zakłócenie). Log mówi, kiedy zmienia horyzont (`KAL: disturbance (|phase| 43 ns) - controller 33 -> 16 s`, `KAL: calm for 120 s - controller 16 -> 33 s`), linia `Learn:` kończy się na `KF`, dopóki tryb działa, a `KL` go pokazuje. Spokojna praca go nie widzi. Zmierzone na płytce autora (26–27.10, skoki `SP` o +82 i +90 LSB): z `KF 16` faza doszła do −58 i −62 ns i była w ±10 ns po 4,4 i 4,0 minuty; z `KF 0` do −86 i −96 ns, pierwszy raz w ±10 ns po 13 i 16 minutach, potem +14 i +12 ns w drugą stronę, na dobre w środku po 25–27 i 32 minutach. Za każdym razem bramka puściła 18–26 s po skoku.

`KL` — odczytaj stan przed, w trakcie i po każdym eksperymencie: estymaty, sigma wiary w fazę, R i Q w użyciu (`[at ceiling]` przy Q znaczy, że adaptacja doszła do swojego ograniczenia `R/KT³` — na stanowisku z nieustającymi mikrozdarzeniami GPS zdarza się to większość nocy i jest własnością nieba, nie usterką), ostatnia innowacja, licznik odrzuceń, **ratio adaptacji** (czy innowacje są większe od przewidywań; >1 naciska Q w górę, <1 w dół — od buildu 71 opisane jako `Q pushed up`, `Q pushed down` albo `consistent`; do buildu 70 `wants more Q` i `wants less`, co brzmiało jak rada co do `KT`, a nią nie jest) i **znaki wodne Q** od startu trackowania (`lo..hi` — czy Q nocą wychodziło z sufitu). Najbardziej informacyjny nawyk: `KL`, potem `SW`, potem godzinny log.

Trendy: `KAL` (normalna), `REJ` (bramka odrzuciła — pojedynczo lub parami przy zdarzeniach GPS to norma), `KEPT` (detektor powtórzył poprzedni odczyt i filtr nie wziął z niego fazy — przy locku mniej więcej co tysięczna sekunda, w trakcie dociągania co trzecia; build 66), `ARM` (uzbrojenie dzielnika — start, szyna lub zastój), `NOPH` (brak fazy, sterowanie z modelu; **cztery sekundy NOPH zaraz po `ARM` to celowa cisza** — rampa czyta wtedy szynę, a to nie jest faza), `NoPL`/`NoCT` (brak kalibracji), `WAIT` (brak danych). Jak wygląda dobra noc — patrz Aneks D.

Część 5 — Wyświetlacze: co znaczy każde pole

Kilka wyświetlaczy może działać naraz (I2C + SPI nie kłócą się). Wszystkie pokazują tę samą prawdę, z różnym poziomem detalu.

5.1 TFT (480x320 albo 320x240)

```
y=  0..23   pasek nagłówka: "GPSDO v1.08"   CPU 58%   LMT 14:32:45 Thu

`CPU 58%` na pasku nagłówka to całkowite obciążenie procesora, zawsze
włączone (pomiar biegnie w zadaniu uptime; `TL` na linii serialowej
dokleja rozbicie per task). Pojawia się od drugiej sekundy po boocie —
okno uśredniania 100 s nie ma jeszcze werdyktu.
y= 30..62   FREQUENCY — wielkie cyfry, kodowane kolorem
y= 70..151  siatka informacyjna, dwie kolumny
y=156..195  rząd czujników
y=204..239  pasek statusu
```

Kolor wielkiej częstotliwości to kontrola zdrowia jednym spojrzeniem:

Kolor	Znaczenie
zielony	zablokowany (dla 10/11: trend `LOCK`; dla 12 również podczas `CORR`/`ZC` — korygująca pętla to zdrowa pętla)
biały	dostraja się
pomarańczowy	holdover
czerwony	brak sygnału / fixa

Podczas procedur startowych liczba jest zastępowana pomarańczowymi odliczaniami: `Survey <s> ±<m>`, `OCXO warmup <s>`, `Tune <s>` (CT), `LTIC cal <s>` (LC), `Calibrate <s>`.

Siatka informacyjna: kolumna lewa — czas UTC + dzień tygodnia, data, uptime (liczony z 1PPS GPS, więc nie dryfuje), `Algo: n <trend>`, kod `PWM:` + napięcie `Vct:`. Kolumna prawa — liczba `Sat:` + `HDOP:` (albo `HDOP: TIME` po survey-in — to słowo jest *dobre*, znaczy tryb czasu na stałej pozycji), `Lat:/Lon:` (6 miejsc), `Alt:` + `qErr:` (wartość piety odejmowana właśnie teraz), `INA:` napięcie i prąd zasilania. Rząd czujników: `BMP:` temperatura + ciśnienie, `AHT:` temperatura + wilgotność, `Vph:/dph:` napięcie detektora i faza w ns (`ovf` = rampa poza ważnym pasem), `Vcc:/Vdd:` szyny zasilania.

Pasek statusu (kolor tła). Zielony znaczy jedno i tylko jedno: pętla regulacji jest zbieżna — na podstawie tego samego werdyktu, który koloruje cyfry częstotliwości powyżej, więc pasek i cyfry nie mogą się różnić.

pasek	kolor	co znaczy
`DISCIPLINED FIX OK`	zielony	fix dobry, pętla w locku
`ACQUIRING FIX OK`	pomarańczowy	fix dobry, pętla pracuje, ale jeszcze nie doszła
`OCXO WARMUP`	pomarańczowy	oscylatora nie ma jeszcze po co oceniać
`CALIBRATING`	pomarańczowy	`C`, `CT` albo `LC` celowo rusza wyjściem
`HOLDOVER (manual)`	pomarańczowy	zamrożone przez `MH`
`HOLDOVER (fix lost)`	czerwony	fix zniknął albo przez 10 s nie było 1PPS (build 68), i pętla zamroziła się sama
`WAITING FOR GPS FIX`	czerwony	nie ma się do czego dyscyplinować

W trakcie survey-in doklejane jest `SURVEY`. Zielony celowo przychodzi wolno, a odchodzi szybko: werdykt pętli jest jednosekundowy i potrafi mrugnąć na pojedynczym zliczeniu licznika, więc pasek chce pięciu kolejnych sekund locka, zanim zzielenieje, i dwóch sekund straty, zanim go odda.

5.2 OLED (128x64) — dwie strony, przełączane co 10 s

Strona A (GPS): czas lokalny, częstotliwość, Lat/Lon/Alt+Sats, uptime, UTC+temperatura AHT, PWM + trend (migający `H/A` na prawej krawędzi = holdover ręczny/auto). Strona B (czujniki): rzędy BMP/AHT/INA, Sat+HDOP, UTC. W trakcie procedur wiersz częstotliwości pokazuje `F SVIN <s>s <m>m`, `F WARMUP <s>s`, `F CAL <s>s`.

5.3 LCD 20x4

Linia 0: częstotliwość; linia 1: UTC + dni uptime; linia 2 rotuje co 10 s (współrzędne / sats+HDOP / AHT / INA / BMP); linia 3: PWM + Vctl + trend z migającym znacznikiem holdover.

5.4 Cyfry zegarowe (TM1637 / HT16K33)

Czas lokalny HH:MM (dwukropki pulsują sekundą). Same kreski = start/brak danych; `oooo` = brak fixa; spinery = rozgrzewka / survey / kalibracja.

Od buildu 67 spinery prawie nic nie kosztują: zadanie wyświetlacza budzi się ok. siedem razy na sekundę, żeby je przesunąć, i przy tych pobudkach dotyka tylko cyfr LED; TFT, OLED i LCD są odświeżane raz na PPS, jak zawsze. Wcześniej każda pobudka przerysowywała wszystko i wskaźnik `CPU` podwajał się na czas rozgrzewki i survey-in (55–56 % wobec 27 % na płytce autora).

Jasność ustawia się przy kompilacji, osobno dla każdego typu wyświetlacza, w `gpsdo_config.h`: `TM1637_BRIGHTNESS` (0-7) i `HT16K33_BRIGHTNESS` (0-15). Żadna nie ma komendy CLI - ustawia się je raz, pod zamontowany moduł i światło w pomieszczeniu.

5.5 Diody na płytce

LED	Znaczenie
Żółta (PB8)	WYŁ = brak fixa GPS *i* brak holdoveru · świeci = fix OK · wolne miganie (1 s) = ręczny holdover, z fixem albo bez · szybkie miganie (200 ms) = utracony fix albo 1PPS, auto holdover
Niebieska (PC13)	**tylko awaria** — szybkie miganie znaczy złapaną usterkę firmware (asercja / przepełnienie stosu / brak pamięci), z powodem na serialu. To *nie* jest pulsometr; ciemna niebieska LED to stan dobry.

Część 6 — Telemetria serialowa (raport 6-liniowy)

Raz na sekundę (raz na PPS) firmware wypisuje blok statusu w tym kształcie tryb human-readable, `RH`):

```
Up: 000d 02:15:33  UTC: 22/8/2026 14:32:45
Lat: 51.477928 Lon: -0.001531 Alt: 46.5m Sat:10 HDOP:TIME
Freq: 10000000.0000 Hz  10s:0.0  100s:0.02  1ks:0.000  10ks:0.0000
PWM:44653  Vctl:1.970V hit
Learn: algo=11 (LTIC-Lars) gain=auto scale=46 phase=12.3ns LOCK qErr=-8.2ns
BMP:23.4C 1013.2hPa  AHT:22.1C 45.3%rH  INA:12.05V 250mA  Vphase:3.077V dph:1390.5ns  CPU:31%
```

(Pozycja w przykładzie to Królewskie Obserwatorium w Greenwich — długość zerowa z definicji, celowo miejsce publiczne. Twoja jednostka będzie oczywiście drukować własne współrzędne; jeśli udostępniasz logi, pamiętaj o opcji **Redact position** w tunerze.)

Linia po linii:

Linia	Pola
1	uptime (liczony z PPS, godny zaufania od v1.06) + data/czas UTC z GPS
2	pozycja (6 miejsc), wysokość (wysokość GPS **plus Twój `A0`**), satelity, HDOP — albo słowo `TIME`, gdy odbiornik timingowy jest w trybie stałej pozycji; brak fixa → `GPS: no position fix yet`
3	suwa liczona częstotliwość (albo `---` przed pierwszym zliczeniem) i uśrednione okna błędu 10 s / 100 s / 1 ks / 10 ks — każde pojawia się dopiero, gdy jego bufor się wypełni
4	kod PWM 16-bit, zmierzone napięcie sterujące, słowo trendu (`hit`, `ACQ`, `DPLL`, `LOCK`, `PLL`, `CORR`, `ZC`, `NOPH`, `NoCT`, `ARM`, `NoPL`, …) — w holdoverze `[HOLDOVER]` zastępuje trend; `RAILS` zastępuje go, gdy pętla idzie od jednego końca zakresu DAC do drugiego bez locka pomiędzy (build 65 — patrz Część 10)
5	linia Learn zależna od algorytmu (niżej) + `qErr`, gdy działa `SAW 1`
6	temperatura/ciśnienie BMP280 (**surowe + Twój `P0`**), temperatura/wilgotność AHT, napięcie/prąd INA, napięcie detektora `Vphase`, faza `dph` w ns (z odjętą piłą, tak jak w samej pętli) oraz `CPU:` — udział ostatniej sekundy, którego procesor nie spędził bezczynnie. Gdy przy starcie znaleziono LM75 (build 68), za nim idzie `LM75:` — temperatura w °C z trzema miejscami po przecinku, `LM75:---`, gdy czujnik przestaje odpowiadać. Przy `TL 1` idzie dalej pole `TL:` z rozbiciem obciążenia na zadania.

Linia Learn per rodzina:

- algo 11: `gain=auto|<wart> scale=<n> phase=<ns> LOCK|acq`
- algo 12: `ph=<ns> level=<n> corr=<n> arm=<n> sig=<ns> zc=<n> secs=<n>` — nagromadzona faza, ostatni działający poziom, korekty, zbrojenia, bieżący szacunek szumu, przejścia przez zero, sekundy
- algo 13: `ph=<ns> f=<ps/s> sig=<ns> R=<ns> rej=<n> arm=<n>`, a w holdoverze `HOLD=<s>`, w czasie zakłócenia `KF` (build 69) — estymaty filtru, nie odczyt z tej sekundy
- algo 10: `state=ACQ|DPLL|LOCK`
- algorytmy 3–9: wyuczony dryf/nachylenie/tłumienie, algo 9 dokleja wyuczony tempco

Tryb tab-delimited (`RD`) drukuje jedną linię na sekundę z tymi samymi danymi jako kolumnami (uptime, okna częstotliwości, sats, HDOP, PWM, napięcia, wszystkie czujniki, surowy TIC) — format do logowania do arkusza. Kolumna ze średnią częstotliwości jest **zostawiana pusta, dopóki jej okno się nie napełni** — odpowiednio pierwsze 10, 100, 1000, 10 000 i 20 000 sekund — żeby wykres pokazywał brak danych, a nie oscylator wskazujący zero herców. Liczba pól się nie zmienia, więc separatory są na miejscu i parser liczący kolumny nie jest naruszony. `RP`/`RR` pauzuje/wznawia. Raport jest nieblokujący: host, który podłączy się i nie czyta, straci ogony raportów, a nie zamrozi wyświetlaczy (naprawione w v1.06 — pełna kolejka CDC potrafiła kiedyś zatkać task wyświetlacza na dobre).

Część 7 — Spis komend (wszystkie)

Serial, 115200, bez rozróżniania wielkości liter, zatwierdzany Enterem. Komendy z argumentem `[val]` **pokazują bieżącą wartość, wywołane bez argumentu**. Obowiązyją dwa reżimy zapisu — firmware zawsze powie w odpowiedzi, na który właśnie trafiłeś:

- **Preferencje zapisują się same** i drukują `[auto-saved: …]`: `TZ`, `TO`, `LT`, `WU`, `SPL`, `SV`, `PO`, `AO` — a ponadto `CT` (sam zapisuje wyprowadzoną grupę PID) i `LC` (sam zapisuje się przy PASS).
- **Parametry pętli żyją w RAM do czasu zapisu** — wszystko, czym obraca tuner: wzmocnienia, parametry LTIC/Lars/algo-12, `LA`, `SP`. Odpowiedź wymienia dokładną grupę do utrwalenia zmiany (`[not saved — run 'ES …']`).

Wersja, pomoc, stan

Komenda	Co robi
`V`	wersja, autorzy, podziękowania — oraz **CRC-32 obrazu flash**, liczone przy starcie z samej pamięci. W odróżnieniu od stempla kompilacji nie może być nieaktualne: builder Arduino używa ponownie plików obiektowych, więc szkic, którego nie ruszałeś, zachowuje starszą datę. Gdy log i pamięć się nie zgadzają, wierz CRC.
`H` / `?`	lista komend · `H TZ` = szczegóły stref czasowych
`SW`	znaki wody stosów, wolny heap, uptime + jego źródło, ppm MCU przeciw GPS — oraz **obciążenie procesora per task**, od największego, uśrednione w prawdziwym oknie 100 s ze stu jednosekundowych kubełków. Mierzone licznikiem cykli Cortexa-M4 przy każdym przełączeniu kontekstu, więc dokładne, a nie próbkowane. Czas przerwań przypada temu zadaniu, które zostało przerwane — czytaj to jako „procesor był tutaj", nie „to zadanie tyle zużyło".
`TL 0|1`	te same liczby per task na linii telemetrii, jako pole `TL:`. Po każdym starcie wyłączone i nigdzie nie zapisywane — to diagnostyka przy stole, nie ustawienie.
(brak)	`RP` / `RR` pauzuje i wznawia telemetrię — jednoklawiszowe TAB/ESC było próbowane i usunięte: prawdziwe terminale i tuner wysyłają linie zakończone CR/LF i nigdy goły TAB ani ESC, więc z narzędzi, z których ludzie faktycznie korzystają, było nieosiągalne.

Wyjście (DAC)

Komenda	Zakres	Co robi
`SP [n]`	1..65535 (bez argumentu = 32767, środek ≈1,65 V)	ustawia DAC sterujący wprost — ręczne sterowanie przy eksperymentach
`up1/up10/dp1/dp10`	—	nudge PWM ±1/±10 (odmawiane w trakcie kalibracji)
`DAC`	`PWM/DITH/EXT`	bez argumentu: raport — ścieżka wyjścia, kody 24- i 16-bit, Vctl **zadane i zmierzone**, jeden krok w µHz (wymaga CT), a na płytce ze zworką zakresu — do której pozycji należy obiekt. Z argumentem: wybiera aktywną ścieżkę wyjścia — **zapisuje się samo**. Sygnał przełącza zworka na płytce; to mówi firmware'owi, którą ścieżką steruje. Nieustawione = `DITH`.
`DV [v]`	2,50..5,50 V	napięcie przy pełnym kodzie, dla "zadane" w raporcie — **zapisuje się samo**. `3.30` (domyślne) to model PWM; na płytce z AD5680 ustaw `5.00`. Zachowywane są trzy miejsca po przecinku (`4.096` dla referencji 4,096 V).
`AV [x]`	1,00..10,00	stosunek dzielnika przed pinem ADC, dla "zmierzone" i każdego wyświetlenia Vctl — **zapisuje się samo**. `1.00` (domyślne) = bez dzielnika.

Komenda	Zakres	Co robi
`VS [VCC|VREF]`	—	do czego zworką podpięty jest dzielnik na PA0 — **zapisuje się samo**. `VCC` (domyślnie) to szyna 5 V, jak na każdej płytce przed V3. `VREF` to referencja napięcia, a raport `DAC` porównuje wtedy odczyt z `DV` i protestuje powyżej 5 %.
`SPAN [CLR FULL|REDUCED]`	—	zworka zakresu EFC na PB14 (`GPSDO_SPAN_SENSE`): bez argumentu = pozycja, w której jest zworka, i kalibracja `CT` (K i zero), którą niesie każda pozycja. `CLR` zapomina kalibrację jednej pozycji — tej zmierzonej w złym miejscu. Raport `DAC` wypisuje te same linie.

Trzy ścieżki, jeden węzeł. Wszystkie trzy ścieżki wyjścia można kompilować razem; komenda `DAC` wybiera, którą firmware pisze, a zworka na płytce decyduje, który sygnał trafia do filtra EFC (nie ma programowego multipleksera — dwa sterowniki na jednym węźle to błąd sprzętowy). Przy starcie każda skompilowana ścieżka jest inicjowana, żeby przełączenie nie zrobiło glitcha, ale krok sterujący pisze **tylko na wybraną ścieżkę**: niewybrany AD5680 trzyma ostatni kod, a gdy żywa jest ścieżka `EXT`, pin PWM jest trzymany nisko (od buildu 69; wcześniej odtwarzał dalej swój ostatni kod, czyli nośną na filtrze PWM). `PWM` to zwykły PWM 16-bit na PB9 (na buildzie z silnikiem ditheru: ten silnik z rozdzielczością całych LSB — to samo napięcie, bez rekonfiguracji). `DITH` to nośna 13-bit + dither sigma-delta 1-bit na PB9/TIM4 CH4, efektywne 24 bity w średniej czasowej (nośna ≈11,7 kHz przy zegarze timerów BlackPilla 96 MHz — `DAC` podaje wartość z płytki — DMA). `EXT` to AD5680 na SPI bit-bang po PB4/PB0/PB2, zapisywany mniej więcej raz na sekundę (~20 µs z zamaskowanymi przerwaniami).

Wewnętrznie wartość sterująca to **jeden kod 24-bit**; wartość 16-bit na wyświetlaczu, w pierścieniu flash i w logach to ta liczba zaokrąglona — widok, nie druga zmienna. Drobne korekty kumulują się w młodszym bajcie (kod 24-bit niebędący wielokrotnością 256 to dowód, że steruje ścieżka precyzyjna; raport `DAC` to zaznacza). Na `EXT` komenda jest mapowana na **18-bitową** część przez skalowanie, nie ucinanie: `code18 = code24 × 262143 / 16777215`. Ramka SPI AD5680 ma 24 bity — `[0000][D17..D0][00]` — z 18 znaczącymi, więc „24-bitowe napięcie sterujące" w dokumentacji oznacza komendę, którą niesie pętla (i którą `DITH` faktycznie realizuje jako średnią czasową); sama część rozwiązuje 18 bitów. Skalowanie sprawia, że pełna skala 16-bit trafia dokładnie w pełną skalę części — dlatego K i wszystkie współczynniki zmierzone na jednej ścieżce przenoszą się na drugą bez zmian. Przy 5,0 V jeden LSB DAC-a to 19,1 µV; na `EXT` referencja *jest* napięciem sterującym, więc jej wolny dryft mapuje się 1:1 na częstotliwość.

Porównanie „zadane przeciw zmierzonym" w raporcie jest uczciwe tylko wtedy, gdy obie skale pasują do płytki: `DV` mówi, co znaczy pełny kod na wyjściu (3,30 V = model PWM, domyślne), `AV` — stosunek dzielnika przed pinem ADC (1,00 = bezpośrednio). Na zewnętrznym DAC-u 5 V z dzielnikiem w torze pomiaru ustawienie obu zamienia check z założenia-płytki-PWM w stwierdzenie o TEJ płytce — bez nich raport będzie flagował MISMATCH przy każdym odczycie. Oba zapisują się same.

Dwa zakresy, dwie kalibracje (`GPSDO_SPAN_SENSE`, PB14). Płytka z przesuwnikiem poziomu EFC może sterować oscylatorem w pełnym zakresie napięcia albo w zawężonym, a to są dwa różne obiekty — na prototypie V3 w pozycji zawężonej K jest około 3,5 razy mniejsze (3,3–3,6 razy, obie pozycje zmierzone na jednej referencji). Podłącz drugi biegun zworki zakresu do **PB14** (zworka założona = PB14 do masy = REDUCED; zdjęta albo niepodłączona = FULL), a firmware będzie trzymał osobną kalibrację `CT` dla każdej pozycji:

- gdy zworka się przestawi (po odfiltrowaniu drgań styku, około pół sekundy), współczynniki pętli są wyliczane od nowa z K tej pozycji, a kod sterujący jest przeliczany tak, żeby pin EFC zachował napięcie — częstotliwość przed przestawieniem jest częstotliwością po nim. Przeliczenie potrzebuje jednej pary kodów, o której wiadomo, że daje to samo napięcie w obu pozycjach; firmware uczy się jej sam — kod, na którym pętla była zablokowana przed przestawieniem, w parze z kodem, na którym zablokuje się potem, albo z kodem 10 MHz wyznaczonym tam przez `CT`. Liczy się tylko kod, który naprawdę jest zerem: nie ten trzymany w holdoverze, nie ustawiony ręcznie (`SP`, `up`/`dp`), dopóki pętla nie straci locka i nie złapie go od nowa, i nie taki, który leży daleko od miejsca, gdzie `CT` znalazło 10 MHz w tej pozycji — musi leżeć w granicach 5e-8 od zera tej pozycji i nie bliżej zera drugiej (w granicach 5e-9 od własnego zera liczy się zawsze). Ten ostatni test łapie pętlę zablokowaną na złym obiekcie, a to właśnie daje stanowisko łączone przewodami, gdy przesuwnik wejdzie w tor albo z niego wyjdzie, zanim PB14 za nim pójdzie. Każda odmowa jest wypisywana. Dopóki pary nie ma, przestawienie zostawia kod, a pętla łapie od nowa;

- pozycja, która nigdy nie była kalibrowana, jedzie na współczynnikach drugiej — jak zawsze jechała płytka z jedną kalibracją — a `CT` startuje samo, gdy tylko jest fix GPS (nigdy w holdoverze), z maksymalnie trzema kolejnymi próbami 10, 20 i 40 minut po nieudanej;
- zworka przestawiona przy wyłączonym zasilaniu jest obsłużona przy starcie, zanim ruszy pętla. Od buildu 65 zapisany kod jest przeliczany ze spanu, w którym według slotu live store został zapisany, a nie ze spanu, który podaje rekord ustawień — to dwa osobne zapisy, a zanik zasilania między nimi po przestawieniu zworki albo rekord, którego zapis się nie udał, potrafiły wystartować oscylator ponad 1e-7 obok.

`SPAN` pokazuje stan, a raport `DAC` wypisuje go pod linią obiektu. Płytka skalibrowana, zanim podłączono PB14, ma swoją jedną kalibrację przypisaną do pozycji, którą pin czytał przy pierwszym starcie z czujnikiem zworki — FULL, jeśli przewodu jeszcze nie było. Jeśli ta kalibracja była naprawdę zrobiona na REDUCED, pierwsze `CT` w drugiej pozycji zmierzy ten sam obiekt, powie o tym i zapomni błędną; `SPAN CLR` robi to samo ręcznie. Ręczne wzmocnienia w LSB (`LG`, `MG`, `LTK`) nie są przeskalowywane — przestawienie ostrzega o każdym, które jest ustawione.

`SPAN` wypisuje też **zero** każdej pozycji — kod, na którym `CT` znalazło 10 MHz, odnawiany przez pary zmierzone od tego czasu — i właśnie z nim porównuje test opisany wyżej. Zera pochodzą wyłącznie z `CT`, więc płytka skalibrowana przed buildem 58 nie ma żadnego: uruchom `CT` raz w każdej pozycji i test działa. `SPAN CLR` zapomina K i zero pozycji, i parę.

Kalibracja

Komenda	Czas	Co robi
`C`	~2 min	starsze dwupunktowe centrowanie PWM
`CT`	~3 min (+8 przy małym K)	**czułość oscylatora K + autostrojenie PID 3–9 i LTIC; sam zapisuje PID** — uruchom najpierw to. Przy łagodnym obiekcie (małe Hz/LSB — zredukowany span EFC, zewnętrzny DAC) drugi przebieg mierzy ponownie wokół dopasowanego kodu 10 MHz: 0,8 Hz huśtawki niezależnie od zakresu, każdy punkt jako średnia ze 100 s, więc K wychodzi z dokładnością około 1 %
`LC`	~5 min	**kalibracja detektora fazy (ns/V, offset, zakres); sam zapisuje się przy PASS** — uruchom jako drugie
`ACG g [cap]`	—	napęd centrowania ACQ: wzmocnienie 50..20000 LSB/V, maks. krok 5..1000 LSB
`AP`	—	ręczne zbrojenie dzielnika picDIV

Tryb i raportowanie

Komenda	Co robi
`RH` / `RD`	raport human-readable / tab-delimited
`RP` / `RR`	pauza / wznowienie raportu 1 Hz
`TL 0|1`	obciążenie CPU per task na linii telemetrii (`SW` pokazuje je raz; belka TFT pokazuje całkowite CPU% zawsze)
`MH` / `MD`	holdover (zamroź PWM, leć sam) / dyscyplina
`F`	przepłucz bufory pierścieni uśredniania częstotliwości
`T [baud]`	przezroczysty tunel GPS na USB pod u-center, 300 s (baud 4800..921600)

Wybór algorytmu i klasyczne PID (patrz Część 4)

Komenda	Zakres	Co robi
`LA [n]`	0..13	wybiera / pokazuje algorytm pętli
`LP [n]`	algo 0..9	wypisuje parametry PID
`KP/KI/KD n val`	algo 3..7, 0..100000	ustawia wzmocnienia

Komenda	Zakres	Co robi
`IL n val`	algo 3..9, 100..100000	ogranicznik integratora
`BC` / `BS`	0,0001..1,0 Hz	punkt przecięcia / skala mieszania algo 8
`NS`	1..10000 LSB	maksymalny krok algo 9

LTIC (algo 10) — zapis przez `ES LTIC`, lista przez `LL`

Komenda	Zakres / domyślne	Znaczenie
`LNV`	0..1e6	nachylenie detektora, ns na wolt (mierzy LC)
`LZO`	0..3,3 V	przesunięcie zera detektora, w woltach
`LRN`	2..1e9 ns albo `0`/`1`/`R`	zakres detektora, ns (mierzy go `LC`; ułamek i wykładnik są w porządku — `LRN 2500.7`, `LRN 1e3`). Ta sama komenda steruje samouczeniem — feed-forwardem dryfu i tłumieniem algorytmów 3–10: dokładnie `LRN 0` je zamraża (to, czego się nauczyło, nadal działa), `LRN 1` wznawia (domyślnie włączone), `LRN R` przywraca dryf i tłumienie do wartości teoretycznych i od razu to zapisuje, samo `LRN` pokazuje stan. Wszystko inne jest odrzucane. `LRN 0`/`LRN 1` zapisuje `ES FLAGS`
`LAT`	0,001..1e9 ns (100)	próg fazy ACQ
`LDT`	1e-13..1,0 (5e-10)	próg dryfu DPLL→LOCK
`LIV`	1..600 s (300)	odstęp korekt w LOCK
`AQP/AQI/AQD/AQL`, `DPP/DPI/DPD/DPL`, `LKP/LKI/LKD/LKL`	0..100000	PID stopni
`LPOL`	−1 / 0 / +1	polaryzacja PWM→faza (0 = nieustawiona: pętla czeka i mówi o tym; `LC` ją mierzy)
`LCV`	0..3,3 V	cel centrowania ACQ
`LRM`	0 / 1 (1), nie zapisuje się	jak czytana jest seria z detektora: 1 = prosta przez 64 pomiary, odczytana na samym zboczu; 0 = mediana z 16, jak do buildu 60
`LB`	—	liczniki od startu (zbocze złapane, prosta odrzucona i dlaczego, odczyt wstrzymany, pominięta trzecia część i przesunięte typowe wygięcie — build 71), potem ostatnia seria: wzięta na zboczu czy później, dlaczego jej prosta została odrzucona, czas i kod każdego pomiaru, prosta, rozrzut, wygięcie i typowe dla płytki, jak daleko od siebie są jej trzecie części, tempo spadku
`FA/FAD/FAL`	10 / 100 / 1000 s	okno uśredniania dla członu tłumiącego (oba / DPLL / LOCK)

LTIC-Lars (algo 11) — zapis przez `ES LTIC`

`LG` (0..10000, 0=auto), `LD` (domyślnie 3), `LTC` (1..600 s, domyślnie 60), `LFD` (1..100, domyślnie 2), `LTO` (0..3,300 V, domyślnie 2,111 V), `LPL` (1..10000 ns, domyślnie 100), `LPF` (1..100, domyślnie 5), `LTK` (±32000, 0=wył.), `LTR` (0..3,300 V) — znaczenia w Części 4.4.

Algo 12 (akumulator wielopoziomowy) — zapis `ES ALGO12`, lista `ML`

`MG` (0..10000 LSB/ns, 0=auto z CT), `MR` (0..10, domyślnie 7), `MM` (0..1, domyślnie 1 = spokojny), `MF` (0..3 źródło limitów, domyślnie 0), `MFT` (0=3600 s, albo 2048..65535 s), `MLP <poziom> <ns>` (poziom 0..10) — znaczenia w Części 4.5.

Algo 13 (Kalman) — zapis w chwili wpisania, bez `ES`

`KR` (0..1000 ns, 0 = mierz), `KQ` (0..1, 0 = adaptuj), `KT` (10..10000 s, domyślnie 100), `KC` (10..10000 s, 0 = auto = KT/3; działa po pierwszym zablokowaniu), `KF` (10..250 s, 0 = wyłączone, domyślnie 16: horyzont na czas zakłócenia), `KL` (wypisz stan filtru) — znaczenia w części 4.6.

GPS, czas, czujniki

Komenda	Zakres	Co robi
`SV 0|1`	—	survey-in / Time Mode wył./wł. (działa od następnego startu) — **zapisuje się samo**
`TZ <miasto|reguła>`	np. `TZ Adelaide`	strefa czasowa z DST (`H TZ` po szczegóły) — **zapisuje się samo**
`TO <h[:mm]|A>`	−14..+14	stałe przesunięcie UTC, albo `A` = auto z pozycji GPS (reguła DST unijna) — **zapisuje się samo**
`LT 0|1`	—	pokazuj UTC / czas lokalny — **zapisuje się samo**
`PO <f>`	−5000..5000 Pa	offset ciśnienia dodawany do odczytu BMP280 — **zapisuje się samo**
`AO <f>`	−3000..3000 m	offset wysokości dodawany do **wysokości GPS** — **zapisuje się samo**
`SAW 0|1`	—	korekta piły (qErr) wył./wł. — samo `SAW` pokazuje stan; zalecane WŁ. przy odbiorniku timingowym (zapis: `ES FLAGS`)
`WU 0|1`	—	rozgrzewka OCXO przy starcie — **zapisuje się samo**
`SPL 0|1`	—	animacja startowa wył./wł. — **zapisuje się samo**
`BL [%]`	30..100	jasność podświetlenia TFT — **zapisuje się samo**. Samo `BL` pokazuje wartość.

Podświetlenie i dlaczego 100 % jest ustawieniem najcichszym. `BL` steruje tranzystorem MOSFET z kanałem P na **PB5** (TIM3 CH2, 20 kHz), włączonym między szynę 3,3 V a anodę LED panelu: bramka przez ~47 Ω, 100 kΩ do masy i 10–47 µF pojemności buforowej przy drenie. Stopień **odwraca** — niski stan bramki to pełna jasność — więc firmware zapisuje dopełnienie, a `BL 100` wypada na porównaniu równym zeru: pin stoi statycznie nisko i stopień w ogóle nie przełącza. Ma to znaczenie, bo to właśnie szyna 3,3 V zasila VDDA, a więc referencję ADC, odczyt fazy i monitor Vctl. Pełna jasność jest ustawieniem najcichszym, nie najgłośniejszym; ściemnianie kosztuje trochę szumu na szynie, a zwraca obciążenie i ciepło w obudowie sprzężonej termicznie z OCXO.

Dolna granica to 30 % z powodu, który nie jest elektryczny: poniżej mniej więcej jednej trzeciej panel przestaje być czytelny, zamiast stawać się użytecznie przyciemniony, więc omyłkowo wpisane `3` zostawiłoby operatora przed czymś, co równie dobrze może być martwą płytką. Na płytce bez MOSFET-a pin po prostu niesie przebieg prostokątny i nic się nie dzieje. `BL` kompiluje się z każdym TFT i **oddaje PB5 generatorowi testowemu 2 kHz**, jeśli ten został jawnie włączony — domyślne ustawienie nie może odbierać pinu świadomej decyzji.

Zapis, odczyt, reset

Komenda	Co robi
`ES [obj]`	**zapisz** wszystko (bez argumentu) albo jedną grupę: `TZ`, `PID`, `LTIC`, `FLAGS`, `ALGO12`, `ALGO`, `PO`
`ER`	**przywołaj** — wczytaj ustawienia z flasha teraz (cofnij niezapisane zmiany)
`EE`	**wymaż** slot ustawień — domyślne od następnego startu
`EW`	zużycie flash ringa: cykle wymazań, zajęte sloty, sektor/adres

Komenda	Co robi
`FR`	status pierścienia tylko do odczytu (zawsze włączony od v0.96)
`CS`	statystyka korekt: liczba, szczyt, RMS ostatnich 100/1k/10k/100k korekt — małe i stabilne jest dobrze, rosnące jest źle
`RB`	ciepły restart (zachowuje ustawienia — ale **nie** zapisuje ich najpierw sam)
`CR YES`	zimny restart: wymaż ustawienia + dane wyuczone, domyślne fabryczne (YES jest wymagany, bo model wyuczony odbudowuje się dniami)

Znane osobliwości v1.08 (uczciwa lista)

W chwili pisania żadne nie są znane. Obie wymienione dla v1.07 — `LRN` czytające argument jako liczbę całkowitą (`LRN 1e3` włączało uczenie, `LRN ON` je wyłączało) i `LRN R` podpowiadające `ES FLAGS`, które nie zapisuje resetu — są poprawione w buildzie 58.

Część 8 — Tuner na PC

`tools/gpsdo_tuner.py` — konsola desktopowa do oglądania i strojenia. Robi wszystko, co terminal, a do tego wykresy na żywo i logowanie CSV.

8.1 Instalacja i start

Python 3.9 lub nowszy, potem:

```
pip install PySide6 pyqtgraph pyserial tzdata
```

(`tzdata` używane tylko przez przycisk *Generate tz_table.h*; na Linuksie/macOS system to ma.)

Uruchomienie: `python gpsdo_tuner.py` (na Windows można też dwuklikiem).

8.2 Połączenie

Wybierz port na pasku, baud 115200, Connect. Tuner natychmiast czyta wersję firmware i **wszystkie** bieżące parametry (`LL`, `FA`, `LP 3`–`LP 9`, wszystkie czasowniki Larsa i algo-12, `ML`) i wypełnia każdą zakładkę — widzisz rzeczywisty stan urządzenia, nie domyślne. Linia stanu pokazuje `state: LOCK (locked)` itd.; tytuły wykresów podążają za aktywnym algorytmem.

8.3 Zakładki

Zakładka	Co edytuje
LTIC (algo 10)	trzy kwartety PID stopni; Read (`LL`) / Save (`ES LTIC`) / Revert (`ER`)
LTIC-Lars (algo 11)	`LG LD LTC LFD LTO LPL LPF LTK LTR` z przyciskami Set
Multi-level (algo 12)	`MG MR MF MFT` + cała 11-wierszowa tabela limitów (`Send limits`, `Read all`, `List (ML)`, `Save (ES ALGO12)`)
FA damping	okna tłumienia DPLL / LOCK
PID algo 3-9	Kp/Ki/Kd/IL per algorytm
Calibration	`LNV LZO LRN LCV LAT LIV LPOL`
Raw monitor	wszystko, co mówi płytka, bez parsowania; sterowanie logowaniem
Help	referencia komend firmware

Pamiętaj, że reguła firmware obowiązuje i tu: **przyciski Set zmieniają tylko RAM; przycisk Save zakładki wysyła `ES ...`, który utrwala.**

8.4 Wykresy

Trzy panele, odświeżane ciągle z telemetrii 1 Hz. Górne panele zależą od rodziny algorytmu — faza (`dph`/`ph`) + napięcie detektora dla 10/11/12 (z szarą kotwicą i czerwonymi krawędziami ważnego pasma, gdy kalibracja jest znana), dryf + napięcie sterujące dla klasyków. Dolny panel to zawsze błąd częstotliwości w Hz. Pasek: wybór zakresu czasu (1 min … all), Follow live, Clear. Rozdzielczość to szybkość telemetrii — zdarzenia szybsze niż ~2 s są niewidzialne, a wykresy ufają płytce.

8.5 Logowanie (wykresy to nie logger)

Raw monitor → **Start logging**, format Full log / CSV only / Both:

- **Full log** `gpsdo_YYYY-MM-DD_HH-MM-SS.log` — każda linia jak wypisana, ~217 MB na tydzień przy 1 Hz.
- **CSV** — sparsowane kolumny: `utc, up_s, algo, state, dph_ns, qerr_ns, vphase_v, pwm, f10, f100, ph_ns, level, corr, sig_ns, zc, bmp_c, sat, hdop` (~65 MB/tydzień). Pusta komórka = pola tej sekundy nie było (nigdy zero).

- **Redact position** (domyślnie WŁ.) czyści Lat/Lon/Alt w zapisanym pełnym logu — dziel się logami bez publikowania własnego dachu.

Część 9 — Ustawienia, flash ring i zużycie

Wszystko, co trwałe, mieszka w jednym **pierścieniu z wyrównywaniem zużycia** we flash sektorze 7: 255 slotów po 512 bajtów. Każdy zapis (`ES`, zdany `LC`, aktualizacja danych wyuczonych) pisze kolejny pusty slot; sektor jest wymazywany dopiero, gdy pierścień zawija się — raz na 255 zapisów — a przy zawinięciu najnowsze rekordy ustawień, Kalmana i danych live są przenoszone za kasowanie (od buildu 65; wcześniej zostawał tylko zapisywany slot i następny start mógł pójść na domyślnych). Przy tempie zapisów tego firmware (nawet zapracowany stole robi ~73 zapisy dziennie) wymazanie wypada co ~3,5 dnia, a flash F411 znosi ~10 000 cykli na sektor: **mniej więcej 96 lat**. Nie zużyjesz tego. `EW` pokazuje realne liczniki, gdybyś był ciekawy.

Dwa rodzaje rekordów dzielą pierścień:

- **Ustawienia** (`ES` i spółka) — wszystko, co wybrałeś, zapisywane tylko na Twoje polecenie. Zapisy grupowe (`ES PID` itd.) piszą *cały* blok ustawień wyseedyowany ostatnim zapisanym — więc zapis jednej grupy nie może nadpisać innej.
- **Dane live** — to, czego urządzenie nauczyło się samo: kalibracja `LC`, model dryfu/tłumienia, ostatni punkt pracy. Zapisywane automatycznie, ale z histerezą (tylko gdy coś realnie zmieniło się o sensowny krok, i najwyżej co 20 minut); `LC`, `LRN R` i przestawienie zworki zakresu zapisują je od razu. Punkt pracy jest zapisywany sam (od buildu 65), gdy odszedł o ponad 0,02 Hz od zapisanego (w LSB przez K z `CT`; bez `CT` 64 LSB), a pętla jest w locku od 20 minut bez przerwy i bez skoku (kod wpisany przez `SP`, `up` albo `dp`, choćby o jeden LSB, albo zostawiony przez kalibrację liczy 20 minut od nowa) — nigdy w holdoverze, w trakcie kalibracji ani w trakcie przestawiania zworki zakresu; algorytmy 0–9 nie mają locka i w ten sposób go nie zapisują. Do buildu 64 trafiał do flasha tylko razem z modelem dryfu, którego algorytmy 11–13 nie ruszają, więc zimny start potrafił zaczynać pół herca obok. Slot mówi też, do którego spanu należy jego kod; slot zapisany przez build 64 nadal się ładuje. Zapis odrzucony przez flash jest ponawiany po 20 minutach.

Aktualizacje firmware: ustawienia z v1.04/v1.05 są migrowane przy przywołaniu (v4→v5 dokładają blok algo-12 z domyślnymi); blok z radykalnie innej wersji albo uszkodzony slot traktowany jest jak „nieobecny" — domyślne i prośba o kalibrację przy następnym starcie. Sam pierścień się leczy: slot na wpół zapisany przez zanik zasilania nie przechodzi CRC i jest po prostu pomijany; wygrywa poprzedni dobry.

Praktyczna checklista, ostatni raz:

- rutynowe wgrywanie: bezpieczne, ustawienia przeżywają (Część 2),
- po sesji strojenia: `ES`,
- przed sprzedażą/podarowaniem jednostki: `CR YES`,
- gdy jednostka dziwnie się zachowuje po sesji eksperymentów: `ER` (przywołaj zapisane, znane-dobre ustawienia), a dopiero potem diagnozuj.

Część 10 — Diagnostyka problemów

Objaw	Prawdopodobna przyczyna → rozwiązanie
Nie kompiluje się, błąd `ltoa`	rdzeń STM32 3.0.0 — zjedź do 2.12.0 (Część 1.1)
Kompiluje się, wypisuje `?` albo śmieci zamiast liczb	Tools → C Runtime Library → **Newlib Nano + Float printf/scanf**
TFT biały po buildzie na rdzeniu 3.0.0	to samo — rdzeń 2.12.0
TFT biały na rdzeniu 2.x	zły sterownik w `User_Setup.h` albo brak `TFT_MISO PA6` (Część 1.4)
Start wisi zaraz po `TFT: init start`	jak wyżej — sprawdź okablowanie i `#define` sterownika
Brak seriala USB po wgrywaniu	naciśnij RESET; jeśli dalej nic — sterownik Zadig dla VID 0483/PID 5740 (Część 2.5)
Ustawienia/kalibracja zniknęły po wgrywaniu	ktoś użył **Erase Chip** — to zasada sektora 7 (Część 2.2); powtórz `CT`, `LC`, `ES`
Żółta LED wyłączona	brak fixa GPS i brak holdoveru — antena, widok nieba, kabel. W holdoverze dioda zawsze miga, więc ciemna dioda nigdy nie oznacza zamrożonego wyjścia
`HDOP:TIME` się nie pojawia	survey-in nieukończony (wymaga odbiornika timingowego + `SV 1` + dobrego nieba); powtarza się po każdym zaniku zasilania
`CT` kończy się porażką / odrzuca K	okablowanie EFC albo czułość oscylatora poza zakresem 0,01–2 mHz/LSB — sprawdź bufor DAC i zakres EFC
`LC` kończy się porażką	najpierw `CT`; nie ruszaj jednostki przez te 5 minut
Trend stoi na `NoCT` (algo 12)	wzmocnienie auto, ale CT nigdy nie biegło → `CT`
Trend stoi na `ACQ` / `NoPL`, algorytmy 10–13	nieustawione `LPOL` (pętla drukuje ostrzeżenie i czeka) — ustaw `LPOL -1` albo `1`, potem `ES LTIC`
Algo 12 koryguje bez przerwy na poziomie 0	zakłócone miejsce — `MF 1` (tabela Alana), patrz Część 4.5
Faza rampuje w dal po `SAW 1`	nie masz odbiornika timingowego / qErr nieważny — `SAW 0`
Linie raportu zamierają, gdy program otwiera CDC i nie czyta	naprawione w v1.06 (zapisy nieblokujące + kolejka 1 KB); jeśli widzisz — działasz na starym firmware
Niebieska LED szybko miga	złapana usterka firmware (asercja / stos / heap) — przeczytaj komunikat na serialu; zgłoś

Objaw	Prawdopodobna przyczyna → rozwiązanie
Jednostka sama się resetuje	sprawdź szynę 3V3 pod obciążeniem OCXO; baner startowy drukuje przyczynę resetu
Wyświetlacz żyje, serialu brak	jesteś tylko na Bluetooth? 57600 na Serial2; albo wpisano `RP` — `RR` wznawia
`LOOP: rail to rail with no lock …` na konsoli, trend `RAILS`	pętla przeprowadziła PWM od jednego końca zakresu DAC do drugiego bez locka po drodze: rozbiega się, a nie dociąga (tak robił algorytm 13 buildu 64 przy pół hercu odejścia; naprawione w 65). `LA 11` dociąga z daleka; jeśli to sam algorytm 11, błędne jest `CT` (K) albo `LPOL`. Bez detektora (brak `GPSDO_LTIC`) linia kończy się „Check CT.". Ostrzeżenie znika przy locku, przy zmianie algorytmu, gdy kod przestawi coś innego niż pętla (`SP`, `CT`, `LC`), albo po 30 minutach bez dotknięcia któregoś końca

Jeśli nic z tego nie pomaga: złap pełny log tunerem (Część 8.5), zanotuj wersję z `V` i pytaj — z dołączonym logiem. Tydzień CSV-a 1 Hz to różnica między zgadywaniem a wiedzą.

10.1 Jak zgłosić problem

Prawie każde pytanie o to firmware dało się rozstrzygnąć czterema rzeczami, a bez nich rozstrzyga się je dniami zgadywania zamiast minutami czytania. Wyślij wszystkie cztery:

1. **Log bootowania** — wszystko od resetu do `GPS init done`, skopiowane jako tekst, nie sfotografowane. Mówi, jaka to płytka, jaki baud GPS wykryto, które czujniki się odezwały, które ramki UBX zostały potwierdzone i jaka była przyczyna resetu.
2. **Twój skompilowany `gpsdo_config.h`** — albo choćby lista przełączników, które zmieniłeś. Większość niespodzianek to różnice konfiguracji, nie usterki: płytka bez detektora fazy, drugi wyświetlacz na tych samych pinach, `SAW 1` na odbiorniku nawigacyjnym.
3. **Jaki moduł GNSS** — prawdziwy u-blox timingowy, prawdziwy nawigacyjny, czy klon (Część 3.2). Klony zachowują się inaczej i sama ta informacja od razu odcina połowę możliwości.
4. **Antena i to, ile widzi nieba.** „W domu na parapecie" to całkowicie dobra odpowiedź i często całe wyjaśnienie.

Jedno sprawdzenie przed wysłaniem: w banerze musi być linia `compiled <data> <godzina>`. Jeśli jej nie ma, masz build sprzed v1.05 i pierwszą rzeczą do spróbowania jest aktualny — kilka zgłoszeń okazało się błędami już naprawionymi. Tę linię drukuje samo firmware, więc nie może się rozjechać z binarką tak, jak zapamiętany numer wersji.

Jeśli płytka **zawiesza się**, dopisz jedną darmową obserwację: czy niebieski LED na PC13 dalej mruga? Przełącza go przerwanie timera 2 Hz i nie zależy od żadnego tasku, więc mruganie znaczy, że MCU żyje, a zablokował się task — to zupełnie inna usterka niż płytka, która w ogóle nie działa.

Aneks A — Jak działa GPSDO, prostymi słowami

Możesz używać urządzenia bez tego aneksu — ale gdy coś zacznie dziwnie zachowywać się, to te dwadzieścia akapitów powie Ci *dlaczego*.

A.1 Problem

Chcesz sygnał 10 MHz, który jest prawidłowy **teraz** (sekunda po sekundzie) i prawidłowy **zawsze** (po miesiącach i latach). Dwa cegiełki dostarczają po połowie i żadna obu:

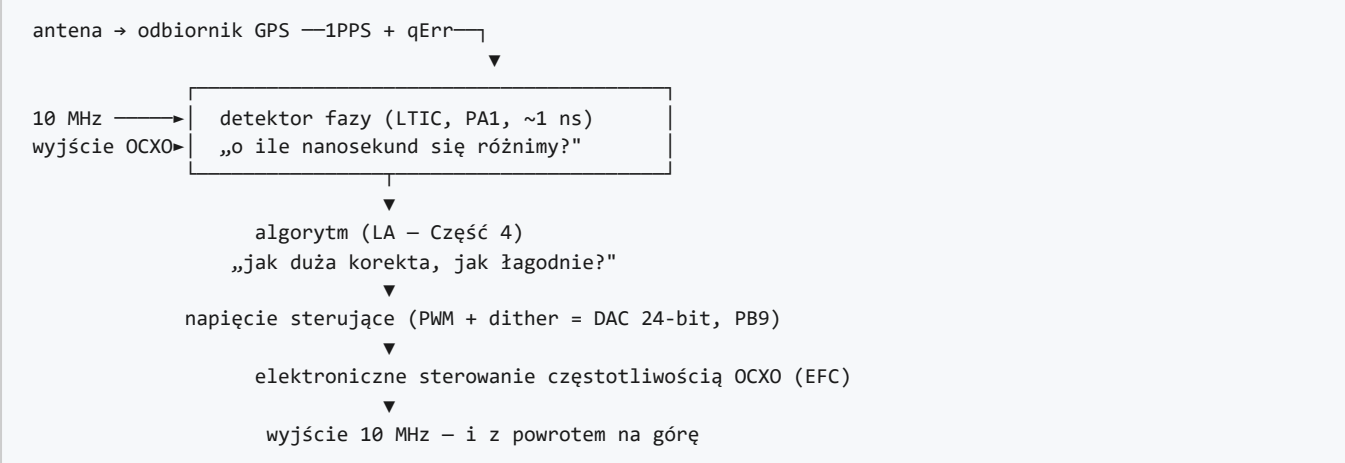
	Krótko (sekundy)	Długo (miesiące)
Sam OCXO	znakomity — dobry trzyma części 10^{11}	dryfuje — starzenie i temperatura powoli odciągają częstotliwość
Sam 1PPS z GPS	szumy — każdy impuls ląduje w ~±25 ns od prawdy	znakomity — sterowany zegarami atomowymi, bez końca

Część „piecu" w nazwie OCXO to pół historii: kryształ siedzi w podgrzewanym pudełku trzymanym na jednej stałej temperaturze (to ta 300-sekundowa rozgrzewka przy starcie), bo temperatura to największy wróg kryształu. Zostaje starzenie — powolny spacer, którego nikt nie wyłączy.

Impuls GPS jest odwrotnością: każda pojedyncza sekunda jest tylko przybliżona (odbiornik kwantyzuje czas, sygnał zwalnia w jonosferze, odbija się od budynków), ale *średnia* z godzin jest przybita do czasu atomowego, bo satelity niosą zegary atomowe i GPS bez tego nie działa.

GPSDO to podział pracy: OCXO obsługuje sekundy, GPS obsługuje miesiące, a pętla sterująca pomiędzy nimi przesuwa tę pierwszą, malutkimi krokami, trzymając ją przypiętą do tej drugiej.

A.2 Pętla, element po elemencie — i gdzie jest co na tej płytce



- **Detektor fazy odpowiada „o ile spóźnieni jesteśmy?"** raz na sekundę, w nanosekundach. Ta jedna liczba — *faza* — to tablica wyników, na której toczy się cała gra.
- **Licznik (TIM2) odpowiada „jesteśmy szybcy czy wolni?"**, licząc dosłownie 10 MHz. Jest gruby (kroki 0,01 Hz), ale bezpośredni.
- **Algorytm** patrzy na te liczby i decyduje jedno: o ile podrobić napięcie sterujące i na jakim czasie rozłożyć szturchnięcie.
- **DAC**: PWM o 65536 krokach, ditherowany do efektywnych 24 bitów (kroki ~0,2 µV — oscylator reaguje na mniej niż jeden krok PWM, więc firmware pamięta ułamek między krokami).
- **Nóżka EFC oscylatora**: wolty na wejściu, herce na wyjściu — cały zakres regulacji ma ledwie kilka herców szerokości, i właśnie dlatego mikrovolty mają znaczenie.

A.3 Faza, częstotliwość i jedyna wymnażanka, jakiej potrzebujesz

Błąd częstotliwości i ucieczka fazy to ten sam fakt w dwóch kostiumach. Jeśli oscylator odjeżdża o ułamek, jego faza ucieka dokładnie w tym tempie:

Błąd częstotliwości przy 10 MHz	Faza ucieka
1 Hz	100 ns co sekundę
0,01 Hz	1 ns co sekundę
0,001 Hz (1 mHz)	1 ns co 10 sekund

Dwie konsekwencje warte wyrycia:

1. **Jeśli faza się nie rusza, częstotliwości są równe** — niezależnie od tego, jaka jest wartość fazy. Prawdziwy cel pętli to *nieruchoma* faza, zaparkowana blisko zera.
2. Błędy, o które warto walczyć, są absurdalnie małe. Jeden miliherc przy 10 MHz to jedna część na 10^{10} — a pętla rutynowo rozróżnia dziesięć razy lepiej. Dlatego każdy przewód, każdy mikrowolt i każdy stopień mają tu znaczenie większe niż w każdym innym obwodzie, który budowałeś.

A.4 Dlaczego korekty muszą być delikatne (serce całego tematu)

Wyobraź sobie, że impuls GPS przyszedł 20 ns za późno, bo sygnał odbił się od budynku. Zmierzony „błąd" fazy to 20 ns — ale nie jest realny: oscylator się nie ruszył. Jeśli pętla skoryguje natychmiast po wartości nominalnej, **skopiuje szum GPS na oscylator** i wyjście będzie *gorsze* niż swobodnie biegnący OCXO. Ten jeden błąd to cała różnica między GPSDO a oscylatorem młóconym przez GPS.

Obrona to uśrednianie: losowy szum maleje jak √N. Uśrednij 100 sekund → szum ÷10; 1000 s → ÷31. Pętla czeka więc, aż uśrednione liczby z GPS staną się cichsze niż własny dryf oscylatora, i dopiero wtedy działa — korektą wymierzoną w to, co długie uśrednienie realnie udowodniło, rozłożoną na porównywalny czas. Dlatego:

- algorytm 11 ma `LTC` (stałą czasową — „jak cierpliwa jest ta pętla"),
- algorytm 12 pozwala **wielkości błędu wybrać własny czas uśredniania** (duży błąd jest udowadniany w sekundy i w sekundy korygowany; mały — przez godzinę i przez godzinę),
- każda korekta w tym firmware jest ditherowana w kroki sub-LSB zamiast wylewania się naraz.

Jeden darmowy obiad: **korekta piły (`SAW 1`)**. Odbiornik *wie*, o ile skwantyzował każdy impuls (liczba `qErr`) i wyznaje to co sekundę. Odjęcie tego wyznania zamienia ±25 ns fałszywego błędu w kilka ns, zanim w ogóle zacznie się jakiekolwiek uśrednianie.

A.5 Holdover

Gdy GPS znika (przecięta antena, martwy odbiornik), pętli brakuje prawdy, którą mogłaby prowadzić. Właściwy ruch to **zamrożenie** ostatniego dobrego napięcia sterującego i pozwolenie OCXO płynąć na własnej stabilności — to holdover (`MH` ręczny albo automatyczny, gdy znika fix albo przez 10 s nie ma 1PPS; żółta LED miga, wyświetlacz robi się pomarańczowy, `[HOLDOVER]` zastępuje trend w raporcie). Oscylator dryfuje wtedy ze starzenia i temperatury — powoli, ale niepowstrzymanie, dopóki GPS nie wróci. Od buildu 68 sprawdzany jest impuls, nie tylko fix, bo odbiornik timingowy w Time Mode podaje swoją stałą pozycję jako ważną bez żadnego sygnału; a automatyczny holdover kończy się dopiero, gdy fix wraca razem z 10 s nieprzerwanych impulsów, więc antena, która łapie i gubi sygnał, nie kończy go impulsem czy dwoma. Ręczny holdover zostaje ręczny mimo takiej utraty (build 68): utrata jest ogłaszana (`GPS fix lost – manual holdover kept` albo `GPS 1PPS lost – …`), a zakończyć go możesz tylko Ty, komendą `MD` — nie powrót GPS.

A.6 Jak ocenia się wyniki: dewiacja Allana w jednym akapicie

Standardy częstotliwości porównuje się **dewiacją Allana (ADEV)**: mniej więcej „o ile średnia nie zgadza się z samą sobą, brana po τ sekund", kreślona przeciw τ. Krzywa GPSDO opada od lewej do prawej: przy $\tau = 1$ s widać własny szum oscylatora; gdy τ rośnie, uśrednianie zbija szum i przejmuje kotwiczenie z GPS (na stole autora: ~$3\cdot10^{-9}$ przy 1 s, spadająco do ~$5\cdot10^{-12}$ przy 3000 s). Gdy ktoś pisze „1e-12 przy tau 10 000", to właśnie to zdanie mówi. *Garb* na środku

krzywej znaczy, że pętla walczy sama z sobą — za szybko jak na własny szum — i to klasyczny podpis źle strojonego GPSDO.

Aneks B — PID dla opornych

Nigdy nie *potrzebujesz* tego aneksu, żeby używać urządzenia — `CT` stroi pętle za Ciebie. To na dzień, w którym otworzysz zakładkę **PID algo 3-9** w tunerze albo wpiszesz `KP` i zechcesz wiedzieć, którą dźwignię trzymasz i w którą stronę gryzie. Nie używa się matematyki ponad mnożenie.

B.1 Obrazek prysznica

Każda pętla sterująca w historii robi te same cztery rzeczy:

1. **Zmierzyć**, gdzie jesteś (temperatura wody).
2. **Porównać** z tym, gdzie chcesz być (komfort).
3. Różnica to **błąd** (za zimno o 5 stopni).
4. **Zadziałać** (odkręć gorący kran), odczekać, powtórzyć.

Jedyny temat całej dziedziny to krok 4: *o ile odkręcić kran?* PID — trzy litery, trzy odpowiedzi — to standardowa recepta.

B.2 P — proporcjonalne: „reaguj na teraz"

Odkręć kran proporcjonalnie do błędu. Zimno mocno → odkręć mocno. Zimno lekko → odkręć lekko.

- **Samo P ma wadę:** potrzebuje pewnego błędu, żeby w ogóle produkować jakiekolwiek działanie, więc osiada *blisko* celu, nigdy na celu — ostatni stopień zimna nie wystarcza, by utrzymać kran otwarty. Ta resztka to **droop**.
- **Za dużo P:** przestrzeliwujesz na gorąco, potem na zimno, i oscylujesz między nimi — prysznic z piekła.
- **Za mało P:** dojeżdżasz wiekową.

B.3 I — całkowe: „pamiętaj przeszłość"

Patrz na błąd *w czasie*. Nawet maleńkie uporczywe zimno kumuluje się, w sumie bieżącej, dopóki pętla nie doda tyle działania, by domknąć. **I zabija droop** — to człon „docelowo, dokładnie".

- **Za dużo I:** klasyczne powolne jo-jo. Suma narasta w podejściu, potem wydaje się przestrzałem, odbudowuje się z drugiej strony i pętla huśta się leniwie w nieskończoność.
- **Windup:** gdy kran jest już pełniutko otwarty (wyjście na ograniczeniu), suma rośnie bezużytecznie i potem wieki się rozkręca z powrotem. Lekarstwo to ogranicznik sumy — i dokładnie tym jest `IL` (I_LIMIT).

B.4 D — różniczkowe: „antycypuj"

Reaguj nie na błąd, tylko na to, jak *szybko* się zmienia. Szybko się ociepla? Zaczynaj przykręcać *zanim* dojedziesz. **D to tłumik** — zawieszenie, które krzyżuje oscylacje, które P i I inaczej by pokochały.

- **Wada D:** wzmacnia szum pomiaru (różniczkowanie drgającego czujnika daje iglice). Dlatego człony D są zwykle filtrowane, małe, albo jedno i drugie — i dlatego szumiący pomiar fazy psuje strojenie obciążone D, a nie poprawia.
- W algorytmach PLL (4/5/7) slot `Kd` działa na nagromadzoną *fazę*, a nie na surową pochodną — ta sama służba tłumienia, inne okablowanie. Nie daj się zmylić literze; myśl „pokrętło tłumienia".

B.5 Trzy litery, jedna tabela

Pokrętło	Reaguje na	Leczy	Za dużo →	Za mało →
P	błąd właśnie teraz	ospałość	przestrzał, szybka oscylacja	dojazd wiekową
I	błąd nagromadzony w czasie	stojący offset (droop)	powolne jo-jo, windup	osiada blisko, nigdy na celu
D	tempo zmian błędu	przestrzał (tłumienie)	nerwowy dryf za szumem	przestrzał dzwoni

B.6 Gdzie siedzą pokrętła w tym firmware

Kręcisz	Gdzie	Którą to naprawdę literą
`KP n val` / `KI` / `KD` / `IL n val`	algorytmy 3–9 (zakładka **PID algo 3-9**)	dosłownie P, I, D i ogranicznik integratora
`AQP…AQL`, `DPP…DPL`, `LKP…LKL`	trzy stopnie algorytmu 10 (zakładka **LTIC**)	pełny zestaw PID na stopień — pętla dostraja się sama w miarę uspokajania
`LG` / `LD` / `LTC`	algorytm 11 (zakładka **LTIC-Lars**)	wzmocnienie (jak mocno), tłumienie (jak osiada), stała czasowa (jak cierpliwie) — te same trzy pokrętła w ubraniach Larsa
`MG` i limity	algorytm 12 (zakładka **Multi-level**)	brak PID — patrz niżej

Algorytm 12 zasługuje na jeden uczciwy akapit: **nie ma P, I ani D**. Zamiast stałych wzmocnień pyta za każdym razem „jak duży jest błąd, jak długo uśredniałem, żeby go udowodnić?" i skaluje korektę do pary pytanie-odpowiedź — duże błędy dostają szybkie, stanowcze traktowanie; małe dostają powolne, delikatne. To ta sama fizyka z wbudowanym strojeniem — dlatego jego jedynymi ręcznymi pokrętłami są wzmocnienie (`MG`, a `CT` mierzy je za Ciebie) i limity rozstrzygające, co liczyc za „duże".

B.7 Dlaczego istnieje `CT` — problem linijki

Liczby PID żyją w jednostkach „kroków PWM na herc błędu". Ale jeden krok PWM jest wart *inną liczbę herców na każdym egzemplarzu oscylatora* — 0,32 mHz na jednym z Vectronów autora, prawie siedem razy mniej na budowie z wąskim EFC. Bez zmierzenia Twojego oscylatora to samo `Kp = 1000` jest delikatnym muśnięciem na jednej płytce i gwałtownym pchnięciem na innej. **`CT` mierzy odpowiedź wolt-na-herc Twojego oscylatora i przeskalowuje każde wzmocnienie do pary** — tak, że fabryczne strojenie znaczy to samo fizycznie na każdej jednostce. Dlatego kolejność kalibracji w tej instrukcji to prawo: najpierw `CT`, potem `LC`, strojenie (jeśli kiedykolwiek) na końcu.

B.8 Dziesięć zasad kciuka

1. Uruchom `CT` i `LC` zanim dotkniesz jakiegokolwiek wzmocnienia. W większości żywotów to koniec strojenia.
2. Zmieniaj **jedno** pokrętło naraz.
3. Potraj albo dziel przez dwa — nigdy ×10. Strojenie pętli reaguje na stosunki, nie na arytmetykę.
4. Daj każdej zmianie godzinę. Te pętle uśredniają po minutach; ocena po trzydziestu sekundach to czytanie książki po jednej literze.
5. Wątpliwości → **wolniej** (większe `LTC`, mniejsze wzmocnienia). Wolno znaczy nudno; szybko znaczy niestabilnie.
6. Oscylacje → tnij P, potem I.
7. Offset, który nigdy do końca nie domyka → więcej I — albo, bardziej prawdopodobnie, pominąłeś `CT`.
8. Wyjście widocznie goni szum GPS → wolniejsza pętla, `SAW 1`, lepszy widok anteny. Nie większe tłumienie.
9. `ES` po każdej sesji; `ER` od-experymentowuje zły popołudnie.
10. Jeśli walczy z Tobą cały dzień — podejrzewaj sprzęt przed strojeniem: widok nieba anteny, temperaturę (grzejniki, drzwi, słońce), okablowanie EFC. Pętle w tym firmware trudno zepsuć, a łatwo obwiniać.

Aneks C — Słowniczek

Pojęcia, które przewijają się przez tę instrukcję, telemetrię i changelog — w znaczeniu, w jakim używa ich ten projekt. Sugestia Alana Cashina, który zwrócił uwagę, że połowa z nich gdzie indziej znaczy co innego.

Pojęcie	Co znaczy tutaj
ADEV	Odchylenie Allana — standardowa miara stabilności oscylatora: o ile zmienia się względna częstotliwość między sąsiednimi oknami uśredniania o długości τ. Czyta się to jak „przy 100 s ten zegar trzyma 1e-11".
ACQ / DPLL / LOCK	Trzy etapy algorytmu 10. ACQ ściąga częstotliwość blisko, korzystając z licznika; DPLL szybko domyka fazę i częstotliwość; LOCK poprawia rzadko i wąskopasmowo, dochodząc do minimum błędu.
BBR	Pamięć podtrzymywana bateryjnie w module GNSS — tam zapisana konfiguracja odbiornika przeżywa zanik zasilania, jeśli jest ogniwo podtrzymujące.
DAC	Przetwornik cyfrowo-analogowy: to, co zamienia liczbę na napięcie sterujące. W tej konstrukcji zwykle nie jest to żaden układ scalony — patrz PWM i dither.
kod DAC / LSB	Liczba zapisywana na wyjście napięcia sterującego i jeden jej krok. Wszystkie wzmocnienia podajemy na LSB, bo to najmniejszy ruch, jaki pętla może wykonać.
dither	Celowe zmienianie młodszych bitów wypełnienia PWM z okresu na okres, tak by *średnia* wypadła między dwoma krokami sprzętowymi. 13 bitów sprzętowych odtwarzanych z tablicy 2048 pozycji daje około 24 bity efektywne.
EFC	Elektroniczne strojenie częstotliwości — wejście strojące OCXO. Wolty na wejściu, herce na wyjściu.
holdover	Praca bez GPS: pętla przestaje korygować, a OCXO biegnie swobodnie na ostatnim napięciu sterującym.
LTIC	Licznik odstępów czasu Larsa — rampowy detektor fazy na PA1 (kondensator ładowany między zboczem PPS a zboczem podzielonego OCXO). Samo „TIC" znaczy ten sam detektor.
NMEA	Tekstowe zdania wysyłane przez moduł GNSS (`$GPRMC,…`): pozycja, czas, liczba satelitów — wszystko poza konfiguracją binarną.
OCXO	Oscylator kwarcowy w termostacie: kwarc trzymany w stałej temperaturze — dlatego jest stabilny i dlatego wymaga rozgrzewki.
picDIV	Mały dzielnik, który z 10 MHz robi zbocze 1 Hz dla detektora fazy. Trzeba go *uzbroić* (zresynchronizować), by jego zbocze padało blisko PPS.
PID / PI	Regulator: **P** reaguje na błąd teraz, **I** na błąd nagromadzony, **D** na to, jak szybko się zmienia. Większość pętli tutaj to PI — patrz Aneks B.
PPS	Wyjście jednego impulsu na sekundę z odbiornika GNSS — wzorzec czasu, względem którego mierzy się wszystko.
PWM	Modulacja szerokości impulsu: fala prostokątna, której wypełnienie po filtrze RC staje się napięciem sterującym. Tanio, a z ditherem lepiej niż większość układów DAC.
qErr / piła	Własne oszacowanie odbiornika, w pikosekundach, o ile jego zbocze PPS spudłowało prawdziwy czas. Odbiorniki timingowe je podają (`UBX-TIM-TP`); korekta usuwa błąd o kształcie piły.
survey-in	Odbiornik timingowy przez długi czas mierzy własną pozycję, by potem trzymać ją na sztywno i cały wysiłek obliczeniowy poświęcić *czasowi*.
Time Mode	Stan, w który odbiornik timingowy wchodzi po survey-in: pozycja ustalona, timing zoptymalizowany. Wyświetlacz pokazuje `HDOP:TIME`.
trend	Czteroznakowe słowo w telemetrii i na wyświetlaczu, nazywające to, co pętla robi w tej chwili: `ACQ`, `DPLL`, `LOCK`, `CORR`, `ZC`, `NOPH`, …
Vctl / Vphase	Vctl to napięcie sterujące idące *do* oscylatora; Vphase to napięcie detektora wracające *z* pomiaru fazy. Dwa różne piny, łatwo pomylić.
ZC	Zniesienie w przejściu przez zero (algorytm 12): zdjęcie celowego slewingu dokładnie w chwili, gdy faza przechodzi przez zero — częstotliwość i faza zostają poprawne naraz.

Aneks D — Filtr Kalmana prostymi słowami

Część 4.6 opisuje algorytm 13 od strony pokręteł; ten aneks od strony intuicji. Bez wzorów — kilka obrazów i tyle.

D.1 Trzy przekonania i ołówek do każdego

Filtr nosi w sobie trzy przekonania: **o ile faza oscylatora jest przesunięta** (w nanosekundach), **jak szybko to przesunięcie rośnie** (pikosekundy na sekundę) i **jak to dryfowanie samo się zmienia z wiekiem** (starzenie). Do każdego przekonania nosi też **grubość ołówka**: przedział, o którym wie, że nie wie. „Faza = 2 ns ± 1 ns" znaczy: jestem pewny na jeden nanosekund — a w ciągu nocy ta pewność sama rośnie i mala.

Ołówki to połowa filtra. Gdy przychodzi pomiar, filtr przesuwa przekonanie **w jego stronę, ale tylko na tyle, na ile grubość ołówka pomiaru pozwala przeciw grubości własnego przekonania**. Pewny pomiar i rozmyte przekonanie — duży krok. Rozmyty pomiar i pewne przekonanie — lekki mus. Cały „filtr Kalmana" to właśnie to: wiara z wagami, odnawiana co sekundę.

D.2 Dwaj świadkowie

- **Detektor fazy** jest świeży, ale gadatliwy: co sekundę mówi o fazie z szumem ~2,5 ns, a do tego jego zero samo wędruje (~2,6 ns w skali ~45 s). Filtr mierzy ten hałas sam — z różnic między kolejnymi odczytami — i nazywa go R.
- **Licznik TIM2** jest uczciwy, ale toporny: częstotliwość mówi z dokładnością, która robi się użyteczna dopiero po stu sekundach. Za to milczy tylko wtedy, gdy milczy wszystko — i to on trzyma pętlę przy życiu, gdy detektor oślepnie.

Krótko mówiąc: na sekundach decyduje oscylator, na miesiącach GPS, a filtr **co sekundę na nowo** wybiera proporcje — zamiast raz na zawsze, jak stała czasowa algorytmu 11.

D.3 Rozum i ręce: KT i KC

Od buildu 36 to dwa osobne pokrętła, i warto rozumieć różnicę:

- **KT** to cierpliwość **rozumu**: jak szeroko filtr rozkłada wiarę w czasie i jak szybki wolno być jego wnioskowaniu (stawia też sufit adaptacji Q — filtr nie może biegać szybciej niż jego horyzont).
- **KC** to szybkość **rąk**: skoro filtr już *wie*, że faza siedzi 5 ns za daleko — po jakim czasie to domknąć? Domyślnie KT/3 (w buildach 61–67 KT/6), ale dopiero **po pierwszym zablokowaniu**: po zimnym starcie pętla wchodzi do pasma spokojnym krokiem KT (od buildu 67 najwyżej 100 s), a szybkie ręce włączą się same, gdy będzie czego szybko domykać. Szybsze ręce to mniejszy *logowany* błąd fazy (ADEV lepsze o ~23% na średnich tau), ale większy dither DAC — budżet, nie darmo.
- **KF** to ręce **w pośpiechu**: gdy coś mocno wybije fazę — spadek zasilania, skok kwarcu — domykają ją w 16 s, aż przez dwie minuty będzie w domu, potem wracają do KC (build 69). Spokojna praca tego nie widzi.

D.4 Zdrowy sceptycyzm, czyli co filtr robi, gdy coś nie gra

- **Bramka 4σ.** Odczyt zbyt dziki jak na własne przewidywania jest odmawiany — pojedynczo i parami przy zdarzeniach GPS to normalna rzecz (noc na buildzie 42: 320 odrzutów w 20 godzin, największa seria dwa). Dziesięć z rzędu i bramka puszcza: błędny jest stan, nie dane.
- **Test zaufania.** Faza i częstotliwość to to samo widowisko z dwóch stron: jeśli licznik mówi, że częstotliwość jest przenoszona, faza *musi* rosnąć. Detektor, który nie rusza się, kiedy musiał — kłamie. Test ignoruje mrugnięcia mniejsze niż własna rozdzielczość licznika (żeby nie skazywał zdrowego detektora na szumie samego wzorca) i pyta, co częstotliwość robi *teraz*: średnia licznika spóźnia się nawet o sto sekund, więc pętla dolicza do niej poprawki, które zrobiła od tamtej pory (build 65 — wcześniej dociąganie mogło skazać zdrowy detektor za ruch sprzed pięćdziesięciu sekund). Skazany detektor dostaje ponowny głos po pół godzinie albo przy najbliższym uzbrojeniu, jeśli jego odczyt skoczy razem z dzielnikiem, czego zamrożony nie potrafi.
- **Cisza po uzbrojeniu.** Uzbrojenie dzielnika zatrzymuje go na sekundę, a rampa czyta wtedy górną szynę (~1400 ns — wygląda jak faza, ale nią nie jest). Przez cztery sekundy po armie filtr słowa „faza" po prostu nie słyszy. Cztery sekundy `NOPH` zaraz po `ARM` w logu to znak zdrowia, nie błędu.

- **Bez odczytu nie ma fazy.** Po restarcie i po uzbrojeniu, a także po dziesięciu sekundach bez odczytu, filtr nie trzyma własnej fazy: steruje samą częstotliwością, dopóki detektor znowu nie zacznie czytać w paśmie. Do buildu 65 niósł fazę zbudowaną z własnej oceny częstotliwości i sterował tak, żeby ją wyzerować; 07.10, przy starcie 0,48 Hz obok i detektorze na szynie, popchnęło to oscylator o 0,8 Hz w drugą stronę. Bez fazy odczyt licznika odrzucony dziesięć razy z rzędu jest przyjmowany, a nie przycinany — skoro nie ma nic innego, odczyt, który uparcie się nie zgadza, ma raczej rację niż filtr.
- **Poniżej okna.** Okno detektora to ±0,55 rozpiętości z `LC` wokół jego zera i wyświetlacz się go trzyma. Algorytm 13 czyta też poniżej, do 0,9 rozpiętości (nigdy poniżej 0,15 V): tam rampa RC jest najbardziej stroma i wciąż jednoznaczna, i tam właśnie ląduje dzielnik po uzbrojeniu. Taki odczyt jest brany za piętnaście razy bardziej zaszumiony niż odczyt przy zerze (build 66 — wcześniej lądowanie poniżej okna nie było żadnym odczytem, a ponowne uzbrojenie ląduje tam, gdzie poprzednie).
- **Brzegi ważą mniej.** Dalej niż ćwierć pasma od zera odczyt jest brany za tym bardziej zaszumiony, im dalej leży — dziesięć razy na brzegu pasma (build 66): tam detektor szumi bardziej, a jego prostoliniowy odczyt jest mniej prawdziwy, i dwa odczyty różniące się o 19 ns tuż po lądowaniu uszły kiedyś za nachylenie 17 ns/s. Pętla w locku tego nie odczuwa: to zaczyna się poza pasmem, które wyznacza `LAT`.
- **Powtórzony odczyt.** Kiedy firmware nie potrafi odczytać serii przy zboczu, publikuje poprzedni odczyt jeszcze raz, a trend pokazuje `KEPT`: filtr nie bierze z niego fazy, ale to też nie przerwa. Przy locku co tysięczna sekunda; w trakcie dociągania co trzecia to norma.
- **Cisza TIM2 po restarcie.** Przez pierwsze sto sekund po starcie stumianowa średnia licznika niesie jeszcze rozgrzewkę oscylatora; filtr czeka na czysty pomiar, zamiast wierzyć w przeszłość.

D.5 Jak wygląda dobra noc

Trend `KAL` przez ~99% czasu. Pojedyncze `REJ` przy zdarzeniach GPS. `NOPH` tylko cztery sekundy po `ARM`, `KEPT` mniej więcej co tysięczna sekunda. R trzyma się ~1,3 ns na płytce autora (noc buildu 64; starsze noce na stanowisku ~2,9), `sig` ~1 ns. Przy Q widoczne `[at ceiling]` — na tym stanowisku mikrozdarzenia GPS naciskają adaptację przez całą noc, więc Q spędza ją na suficie; to własność nieba, nie usterka. `KL` po nocy dorzuca ratio adaptacji i znaki wodne Q (`lo..hi`) — z nich widać, czy Q nocą kiedykolwiek zeszło z sufitu.

D.6 Kiedy niczego nie ruszać

Domyślne KR/KQ/KT/KC są **mierzone z samej płytki** — R z jej detektora, Q z jej oscylatora, resztę z horyzontów. Przypinanie (KR do podłogi białej, KQ na zasiewie) służy eksperymentom porównawczym, nie codziennej pracy: każde przypięcie mówi filtrowi, że wie lepiej niż płytką — a płytką zwykle nie kłamie. Jeśli chcesz zobaczyć, *czy* filtr ma rację, najlepszym pojedynczym nawykiem pozostaje: `KL`, `SW`, godzinny log — i Aneks A, żeby wiedzieć, czego patrzeć.